



National University of Computer and Emerging Sciences, Lahore



ParkEase: Smart Parking System

Zain Ahmad	20L-1011	BS(CS)
Ali Raza	20L-1005	BS(CS)
Syed Hassan Haider	20L-1152	BS(CS)

Supervisor: Ms. Abeeda Akram

Final Year Project

June 5, 2024

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: ParkEase: Smart Parking System

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: October 18, 2023

Name: Zain Ahmad

Signature:



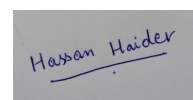
Name: Ali Raza

Signature:



Name: Syed Hassan Haider

Signature:



Author's Declaration

We hereby declare that the work presented in this report is our own and has not been submitted or presented previously to any other institution or organization.

Abstract

Managing the quality of parking in the city is an ongoing challenge that impacts driving and parking management, further amplified with the increase in vehicle ownership leading to traffic in congestion. This project brings a new approach to parking management by combining security cameras and computer technology. Through the integration of computer vision, our system can monitor parking facilities for free spaces and inform the drivers for convenient parking. Other responsibilities include directing drivers to the nearest stop to prevent accidents, monitoring accidents, fires, violence and other violent incidents. This system not only benefits the drivers but the parking lot administration gains valuable data-driven insights into parking usage, allowing them to improve allocation and improve overall management. Drivers can experience a more efficient, user-centered approach to parking, improving the overall experience. This project is an important step in solving the city's parking problem and paves the way for smarter and easier parking. Solve parking problems more easily. Help the city and its residents.

Executive Summary

Drivers sometimes struggle with the inconvenience of personally verifying each sensor in traditional parking systems that rely on sensor technology to determine whether a parking place is available. In addition to taking a lot of time, this procedure raises the installation and maintenance expenses for institutions significantly. Conversely, we provide an effective technology called ParkEase.

ParkEase is an innovative program that uses strategically placed security cameras within parking facilities to optimize parking lot management and improve security measures. It makes the system much cost friendly as we use the Computer Vision technology integrated with the CCTV cameras. These cameras keep an eye on parking spots all the time, determining whether cars are there or not in real time. ParkEase reduces the amount of time drivers spend looking for a place by providing instantaneous updates on available parking spots via an intuitive display as soon as they approach the parking entry. This eliminates the need for manual checks. In addition, our technology optimizes customers' parking experiences by recommending the quickest path to the closest available parking space, adding still another level of convenience. In addition to streamlining the parking procedure, ParkEase offers establishments looking for a more affordable option.

Utilizing Convolutional Neural Networks (CNNs), our system detects and monitors empty parking spaces in real-time. It will detect and update the empty spots when a car reaches the entry barrier or exit barrier. A digital screen at the parking lot entrance displays the available parking spots, allowing drivers to make informed choices upon entry. Also, that screen will show the shortest path to the nearest empty spot. ParkEase calculates and displays a one-time optimized route to the chosen parking spot, minimizing congestion and frustration. The system employs computer vision to detect collisions, break-ins, and other such altercations within the parking area, providing an added layer of security. To enhance security feature, it will notify the management of any unusual activity happening in parking lot.

ParkEase aims to revolutionize the parking experience for not only the drivers but also the management as well. Drivers do not have to spend time looking for parking spots which results in reduced traffic and congestion in the lot, time and fuel savings. For the management, ParkEase offers a convenient way to keep track of the entire lot at a glance. Keeping track of how much traffic they get in different times of the day, aiding the management in making informed decisions about expansion and dynamic pricing based on traffic if they so choose.

In conclusion, ParkEase represents a significant advancement in parking lot management and security. By utilizing Computer Vision technology, we offer an intelligent solution that benefits both drivers and parking facility operators. Our system not only simplifies the parking experience but also ensures the safety and security of all stakeholders.

Table of Contents

List of Figures	x
List of Tables	xi
1 Introduction	1
1.1 Purpose of this Document	2
1.2 Intended Audience	2
1.3 Definitions, Acronyms, and Abbreviations	2
2 Project Vision	4
2.1 Problem Domain Overview	4
2.2 Problem Statement	4
2.3 Problem Elaboration	5
2.3.1 Data Collection	5
2.3.2 Data Pre-processing	5
2.3.3 Parking Spot Classification	6
2.3.4 Optimal Routing	6
2.3.5 Crime Detection	6
2.4 Goals and Objectives	6
2.5 Project Scope	7
2.6 Sustainable Development Goal (SDG)	7
3 Literature Review / Related Work	8
3.1 Definitions, Acronyms, and Abbreviations	8
3.2 Detailed Literature Review	8
3.2.1 Parking Spot Classification	9
3.2.2 Optimal Route Planning	11
3.2.3 Crime Detection	14
3.2.4 Relevant Applications	17

3.3	Literature Review Summary Table	19
3.4	Conclusion	20
4	Software Requirement Specifications	21
4.1	List of Features	21
4.2	Functional Requirements	21
4.3	Quality Attributes	22
4.4	Non-Functional Requirements	22
4.5	Assumptions	23
4.6	Use Cases	23
4.6.1	System Setup	23
4.6.2	System Start	24
4.6.3	Security Alerts	24
4.6.4	Vehicle Information	25
4.6.5	Camera Feed	25
4.7	Hardware and Software Requirements	25
4.7.1	Hardware Requirements	26
4.7.2	Software Requirements	26
4.8	Graphical User Interface	26
4.9	Database Design	31
4.9.1	ER Diagram	32
4.9.2	Data Dictionary	33
4.10	Risk Analysis	34
4.11	Conclusion	35
5	Proposed Approach and Methodology	36
5.1	Parking Lot Classification	36
5.1.1	Data Collection	36
5.1.2	Model Details	36
5.2	Optimal Route Planning	37
5.2.1	Techniques	37
5.3	Crime Detection	37
5.3.1	Data Collection	37
5.3.2	Data Pre-Processing	37
5.3.3	Model Details	38

6	High-Level and Low-Level Design	39
6.1	System Overview	39
6.1.1	Parking Spot Classification	39
6.1.2	Optimal Routing	39
6.1.3	Crime Detection	39
6.1.4	License Plate Recognition	40
6.2	Design Considerations	40
6.2.1	Assumptions and Dependencies	40
6.2.2	Software Requirements	40
6.2.3	Hardware Requirements	40
6.2.4	General Constraints	40
6.2.5	Goals and Guidelines	41
6.2.6	Development Methods	41
6.3	System Architecture	42
6.3.1	Subsystem Architecture	42
6.4	Architectural Strategies	44
6.4.1	Programming Language	44
6.4.2	Future plans for extending or enhancing the software	45
6.4.3	Concurrency and Synchronization	45
6.5	Domain Model/Class Diagram	45
6.6	Sequence Diagrams	46
6.7	Policies and Tactics	51
6.7.1	Coding guidelines and conventions	51
6.7.2	User Interface	51
6.7.3	Extensibility	51
6.7.4	System Testing	52
6.7.5	Engineering Trade-offs	52
7	Implementation and Test Cases	53
7.1	Parking Spot Classification Model	53
7.1.1	Custom CNN Approach	53
7.1.2	cmAlexNet Approach	53
7.2	Crime Detection Model	54
7.3	Automatic Number Plate Recognition	54
7.4	Spot Information	55

7.5	Vehicle Information	55
7.6	Set Credentials	55
7.7	Select Cameras	55
7.8	Mark Parking Spots	55
7.9	Route Calculation	56
7.10	Test Case Design and Description	56
7.10.1	Parking Spot Classification	57
7.10.2	Anomaly Classification	58
7.10.3	Automatic Number Plate Recognition	59
7.10.4	Setup Screen	61
7.10.5	Start Screen	63
7.11	Test Metrics	64
8	User Manual	65
8.1	System Setup	65
8.1.1	Camera Selection	65
8.1.2	Parking Spot Segmentation	65
8.2	Start System	65
8.2.1	Camera Feed	65
8.2.2	Spots Information	66
8.2.3	Vehicle Information	66
8.2.4	Security Alert	66
9	Experimental Results and Discussion	67
9.1	Evaluation Metrics	67
9.1.1	Accuracy	67
9.1.2	Precision	67
9.1.3	Recall	68
9.1.4	F1-Score	68
9.1.5	Binary Cross Entropy	68
9.2	Experimentation Setup	68
9.3	Results	68
9.3.1	Parking Spot Classification	69
9.3.2	Crime Detection	71
10	Conclusions	73

List of Figures

2.1	Sustainable Development Goal	7
3.1	Nazia et al.[1] Reported Accuracy	10
3.3	Results proposed by Jujin and G. Xiaoxue[2]	12
3.4	Talha et al.[3] proposed results	15
3.5	Ijjin et al.[4] Reported Accuracy	15
3.6	Ghahremannezhad et al.[5] Proposed Results	16
4.1	Start Screen	27
4.2	Live CCTV Feed Screen	27
4.3	Parking Slots Information Screen	28
4.4	Vehicle Record Screen	28
4.5	Vehicle Record Screen - Collapsed NavBar	29
4.6	Security Alert Screen	29
4.7	Outdoor Camera Selection Screen	30
4.8	Indoor Camera Selection Screen	30
4.9	Segmentation Screen	31
4.10	Driver Information Screen	31
4.11	ER Diagram	33
5.1	Flow Diagram	38
6.1	Architecture Diagram	42
6.2	Architecture of Parking Classifier	43
6.3	Architecture of Crime Classifier	44
6.4	Class Diagram	46
6.5	Sequence of System Setup	47
6.6	Sequence of Route Update	47
6.7	Sequence of Vehicle Entry/Exit	48
6.8	Sequence of Parking Classification	49

6.9	Sequence of Displaying Vehicle Record	50
6.10	Sequence of Crime Detection	51
9.1	Confusion Matrix for Test Results Custom CNN	69
9.2	Confusion Matrix for Test Results cmAlexNet	70
9.3	Confusion Matrix for Test Results Crime Detection	71

List of Tables

1.1	Price Estimate Comparison of Our Solution	1
3.1	Review of the Research Papers	19
3.2	Summary Table of Similar Applications	20
4.1	Data Dictionary Table	34
7.1	Parking Spot Classification Test Case	57
7.2	Anomaly Classification Test Case	58
7.3	License Plate Localization Test Case	59
7.4	License Plate Recognition Test Case	60
7.5	Camera Selection	61
7.6	Spots Segmentation	62
7.7	Camera Feed	63
7.8	Vehicle Information	64
7.9	Test case Metrics	64
9.1	Evaluation Metrics Performance Custom CNN	69
9.2	Evaluation Metrics Performance cmAlexNet	70
9.3	Evaluation Metrics Performance Crime Detection	72

Chapter 1 Introduction

Finding a parking spot is one of the major problems in metropolitan areas and with the increase in population and the purchase of vehicles, this problem is on the rise. In addition to being a major time wastage, driving around for prolonged time to find a viable parking spot can have an adverse effect on the environment contributing to pollution and also end up being a strain on the monetary resources of car owners by wasting fuel.

Thus, many solutions have been proposed to solve this problem in a timely and cost-effective manner. These solutions include installing magnetic sensors in the concrete, or using radar, ultrasonic sensors above or below the cars. Use of ultrasonic sensors is by far the most popular solution being used today. But due to the high installation and maintenance cost associated with this solution, it is limited mostly to large indoor parking areas like at the mall or an airport or a parking plaza. Henceforth, the need for a more widely available solution still remains.

ParkEase is one such solution which aims to bridge the gap between acceptable performance and the associated running cost of the system. We aim to propose a system for managing parking lots which can utilize security cameras to monitor parking spots as compared to sensors. As most parking lots have security cameras, this solution will be as simple as running a program. For new parking lots, we aim to bring down the total installation cost of the system from around PKR 3.5 million¹ to as low as PKR 200,000². The table below gives a breakdown of the cost analysis and comparison of our solution with current solutions.

Table 1.1: Price Estimate Comparison of Our Solution

This table compares the price estimates for a parking lot with 100 spaces. The estimates were taken by contacting NedoGroup and PakGerman Engineers.

	<i>Our Solution</i>	<i>Traditional Sensor Based Solutions</i>
Hardware/Software Requirements	20-25 Cameras Switch/Router Central server/computer Central display	100 Ultrasonic Sensors Zone Control Unit Parking Guidance Software Display per zone
Price Breakdown	5000 PKR per Camera 7500 PKR per 24 port switch, 2 needed Variable cost roughly 100,000-150,000 PKR Price dependant on size	16,000-20,000 PKR per sensor and 3,500 PKR installation per sensor 150,000 PKR per unit, 3 Needed 650,000 PKR 105,000 PKR per unit, 2-3 Needed
Total Price	Roughly 350,000 - 400,000 PKR	Roughly 3.5 Million PKR

¹Price estimate for a parking lot with 100 spots. Estimates are taken from <https://pgepakistan.com/> and <https://www.nedogroup.com/>. Estimate provided is PKR 350,000 per ultrasonic sensor.

²Estimate including 20-30 CCTV cameras and a Computer for inference and processing.

1.1 Purpose of this Document

This document provides not only the background on the research done in this field but also provides an overview on how we plan to develop our system.

The system we plan to develop aims to solve the problem of parking lot classification by providing a desktop application with a user-friendly design which will connect with the cameras installed in the parking lot and apply various computer vision and/or deep learning techniques for detection. The system will identify the occupied and available spots in the lot and display them on a screen which would be installed at the entrance of the lot to inform the drivers about the state of the parking lot. To reduce the energy and bandwidth requirements, rather than processing the entire video feed all the time, the system will intelligently process frames at specific intervals determined by the entering or leaving of vehicles. To track cars entering and leaving the lot, the system will employ OCR techniques to scan license plates of vehicles. To provide additional benefits, the system will also utilize the same security camera feed to detect any anomalies in the parking lot. These can include theft, accidents, fire, vandalism and so on.

The main research questions we aim to answer with our project are: Can vision based solutions be considered a suitable replacement for sensor based detection based on the price difference? Can the system be generalized to work on a variety of conditions like camera angles, weather and lighting etc. This report will present the methodology, design, implementation, testing, and evaluation of our project, as well as the limitations and future work.

1.2 Intended Audience

This document is intended for the FYP panel and our supervisor to analyze and evaluate our approach. This can also be utilized by technical personnel as a starting point in building a project similar in functionality to this as we look at current research and provide possible implementation and design techniques that can be followed to build the system.

1.3 Definitions, Acronyms, and Abbreviations

In the interest of clarity and readability, this report utilizes several acronyms. The following is a list of these acronyms along with their full forms:

- **CNN:** Convolutional Neural Network
- **YOLO:** You Only Look Once - An image classification Model
- **RCNN:** Region Based Convolutional Neural Network

The second section of the report highlights in detail the problem we aim to solve, and how our system will solve the specified problem. The third section details current relevant research in the areas related to our project. These include parking spot classification, anomaly/crime detection, and parking lot routing techniques.

Chapter 2 Project Vision

The project provides an innovative solution to address the persistent challenge of efficient parking facility management in urban areas. Through the integration of security cameras and computer vision technology, system aims to revolutionize real-time monitoring of parking lot occupancy and vehicle movement. By tracking license plates of vehicles, the system provides precise entry and exit time recording. This gives data insides for parking administration. The goal is to give functionalities such as significant information, resource allocation and parking management to the administration. This will result in efficient parking management.

2.1 Problem Domain Overview

In cities, both drivers and parking authorities face challenges in managing parking. There's not enough parking, making it stressful for drivers. Parking authorities find it hard to keep track of how long cars stay and how much money they make. Right now, they use expensive ultrasonic sensors. Our solution is a new app that uses security cameras and computer vision to solve these problems. This innovative approach makes parking management smoother. It helps management make smart decisions using real-time data, making things better for drivers.

2.2 Problem Statement

Finding parking in cities is hard for both drivers and parking authorities. It takes a lot of time and is stressful for drivers. Parking authorities also have a tough job monitoring how full the parking lots are, how long cars stay, and how much money they make. Right now, they spend a lot of money, around PKR 20,000, just to track if one parking space is taken, using things like ultrasonic sensors. This proposal addresses this issue by introducing a novel approach leveraging security cameras and computer vision technology. Through real time monitoring of parking lot occupancy and vehicle movement, facilitated by computer vision algorithms. The system aims to provide accurate insights. Moreover, the incorporation of license plate recognition allows for precise recording of entry and exit times. This data driven approach empowers parking authorities with valuable insights to optimize resource allocation and enhance overall management efficiency. This will ultimately contribute to an improved urban parking experience.

2.3 Problem Elaboration

Our proposed system aims to tackle the proposed problem by implementing a system that contains the following possible modules.

- Parking spot classification module
- Shortest route planning module
- Anomaly detection module
- Vehicle entry/exit tracking module
- Display update module

For easier understanding and future implementation, we have divided the problem into following sub problems:

2.3.1 Data Collection

Different data is needed for different modules to function properly. For parking classification, we would need to collect dataset of various parking lots in various lighting and weather conditions. To make the system more generalizable, the ideal dataset should contain images at various heights, camera angles and different parking settings like indoor and outdoor. For anomaly detection, we need dataset containing various crimes or incident occurring in the parking lot. This dataset should contain images related to various common crimes in parking lots like theft, vandalism, accidents, fires and others. Similarly, ideal dataset should contain various images related to various conditions to make the system more general.

2.3.2 Data Pre-processing

Once the data is collected, it will undergo a pre-processing stage. The pre-processing stage will differ for different types of data like images and videos.

When dealing with videos, pre-processing means cleaning up the video and changing it into a format that we can study more easily. We use techniques to reduce any unnecessary stuff in the video, making it better. We also take each part of the video and turn it into separate pictures so our computer programs can understand it better.

For pictures we will use techniques like segmentation, labelling, and resizing. For model generalization, data augmentation techniques can be used such as image transformation, rotation, noise.

2.3.3 Parking Spot Classification

After getting the pictures ready. Our system looks at them to decide if each parking spot is occupied or empty. It will use special computer vision techniques. This real time check on which parking spots will help us to manage the parking lot more efficiently.

2.3.4 Optimal Routing

Our system will also guide the driver, optimal route to the empty parking spot. This will make the process more quick and less stressful. Route planning will be done using different route optimization algorithms such as A-star and Dijkstra's.

2.3.5 Crime Detection

Our system will also detect unusual activities in the lot. It will keep track of vehicle total time spent in lot. This will be done using licence plate recognition at entry and exit point. This information combined with other data can be used to detect any unusual events happening in the parking lot. We are figuring out the exact techniques for this purpose. You can find more details in the third section of this report.

2.4 Goals and Objectives

Our project objectives include:

- Security cameras to watch all the parking spaces in the lot. This will be done using computer vision techniques.
- Using optimal route planning algorithms to tell drivers about shortest path.
- Using crime detection algorithms to spot unusual activities in the parking lot.
- Train an OCR model to read the license plate, this will also be used to keep track of the total time spent by vehicle in the lot.
- Map will be displayed at the entrance of the lot showing optimal route, occupied and empty spots.
- A desktop application managing all these functionalities will be deployed on the parking administration side.

2.5 Project Scope

Our system uses special computer tricks to see which parking spaces are taken and which ones are free, replacing the sensors in other parking systems. It can also read license plates when cars come in and leave, so it knows how long each car stays. This helps with security and can even detect if something suspicious is happening. After figuring out the parking spots, the system can guide drivers to the closest free spot. Users can also make a map of their parking lot that shows up at the entrance. The map will also highlight empty and occupied spots so drivers can make an informed decision about parking when entering the lot. Finally, a user-friendly desktop application with a GUI will be created to help the management navigate and use the system without hassle.

2.6 Sustainable Development Goal (SDG)

The project described aligns with Sustainable Development Goal 9: Industry, Innovation, and Infrastructure. Our system assists urban communities with utilizing assets better and makes it more straightforward for everybody to be a part of the cycle. The proposed parking management system, incorporating security cameras, computer vision, and data-driven insights, contributes to the development of innovative solutions for urban infrastructure management. This will enhance the efficiency and resource utilization in the context of urban transportation and parking facilities.



Figure 2.1: Sustainable Development Goal

Industry, Innovation and Infrastructure as our SDG due to innovative solution proposed.

Chapter 3 Literature Review / Related Work

This chapter presents the detailed literature review of studies relevant to parking space classification, crime detection and parking lot routing using computer vision and some relevant applications. Two summary tables are provided at the end to summarize the findings.

3.1 Definitions, Acronyms, and Abbreviations

This chapter makes use of several acronyms for ease of reference. To ensure a clear understanding of the terminology used throughout the chapter, a list of acronyms and their respective meanings are provided in the list below:

- **CNN**: Stands for Convolutional Neural Network. It is a deep learning model which uses a regularized feed forward neural network capable of self learning using filters. It is mainly used of image classification tasks.
- **AlexNet**: Proposed by Alex Krizhevsky in 2012. It is an 8 layer deep neural network commonly used for image classification tasks due to its high accuracy.
- **GoogleNet**: A 22 layer deep CNN released by Google.
- **Raspberry Pi**: A single board computer popularized due to its compact size, open design, modularity and low cost.
- **CNN BiLSTM**: A convolutional neural network combined with a bidirectional LSTM network.
- **R-CNN**: A deep learning technique that focuses on object localizing using rectangular region proposals with CNN.
- **YOLOv4**: Stands for you only look once. A powerful and efficient object detection model that strikes a balance between speed and accuracy.
- **RFID**: Radio Frequency Identification. Technology used for the identification and tracking of objects, people, or animals using radio waves.

3.2 Detailed Literature Review

This section offers a comprehensive review of studies that are relevant to our project. It details the datasets used, the techniques applied, the results obtained, and provides a critical analysis of these studies. The relevance of these studies to our project is also discussed.

3.2.1 Parking Spot Classification

Finding vacant parking spots in a lot continues to be a sore for drivers around the world. Despite the technological advancements, this problem is yet to be remedied. The first and most important part of this problem is classifying the parking spots as either empty or occupied. Many solutions have been proposed for this including using magnetic sensors, ultrasonic or radar sensors. With the recent advancements in the field of computer vision, many camera/vision based solution have also been proposed to classify the parking spots. These solutions range from using myriads of computer vision techniques or using deep learning models for the classification.

Though no datasets are needed to train the model for classification techniques based on simple computer vision, using modern deep learning techniques does require datasets for training and evaluating the model. Throughout our research, we have only been able to find two publicly available parking datasets which are PKLot[6] and CNRPark-EXT[7]. However, these datasets only contain images for outdoor parking lots. Thus, to further generalize the classification model, a dataset for indoor parking lot needs to be created. This can be done by capturing a video of a parking lot from various angles, at different times of the day. The video can be converted into frames. To further increase the size of the dataset, which would help with generalization, data augmentation techniques like transformation, rotation, adding noise etc can be employed.

Using computer vision based solutions can pose an advantage as there is no training data needed or time required to train the neural network. One such solution is also proposed by Nazia et al.[1] which utilizes some computer vision techniques to classify the parking lot as either available or occupied. The proposed solution, for the first time during setup, takes input the coordinates of all the parking spaces in the lot. Then, the system captures frames from live feed at specified intervals and converts them to gray-scale. Finally the system uses block to binary and inverse binary color transforms to convert the car and the marked spot into white and black colors respectively. It then calculates the amount of white pixels and compares it with a specified threshold to classify the spot. The accuracy of the algorithm is reported in figure 3.1.

	<i>Vehicle Appearance</i>	<i>Number of tests</i>	<i>Correct detection</i>	<i>False detection</i>	<i>Accuracy</i>
Sunny day	Clear	50	50	0	100%
	Occluded	50	48	2	96%
Cloudy day	Clear	50	49	1	98%
	Occluded	50	46	4	94%

Figure 3.1: Nazia et al.[1] Reported Accuracy

This table shows the accuracy reported by the proposed model in different weather and lighting conditions.

A deep learning based approach is proposed by Roland et al.[8] and Sayuti et al.[9]. A custom CNN created using Tensorflow Lite is used by Roland et al.[8]. The CNN consists of 9 convolutional layers and 1 fully connected layer and uses a kernel size of 5 for the first convolutional layer and 3 for remaining layers. The neural network is trained using PKLot[6] and CNRPark+EXT[7] dataset. Initially, a separate module is used to mark the parking spots. Then these spots are fed into the CNN to classify each spot. The accuracy of the model reported is 93.79%. When classifying an image of 14 spots, processing takes 0.21876s on Raspberry PI, averaging around 0.014s per spots. The research proposes that using a single Raspberry PI, the model can classify upto 66 spots in real time.

Though Sayuti et al.[9] have also proposed a deep learning based solution for the problem, the difference is in the architecture of the CNN used. The proposed model utilizes a slightly modified version of the mAlexNet which itself is a modified version of the AlexNet. The main modification made to the mAlexNet by the researchers is an increase in the number of filters and size of filters from 11x11 to 13x13 in the first convolutional layer only. For training, CNRPark[7] dataset is used again. The machine used for training and testing has 4GB ram and an i3 processor @3.30Ghz. CNN is trained for 5-6 epochs using Matlab. The research proposes an average accuracy of 99.12% outperforming other larger pre-trained neural networks like GoogleNet and AlexNet. The complete results are as follows:

3.2.1.1 Critical analysis of the research item (Strengths and Weaknesses)

Despite reporting high accuracy, all the proposed solutions have inherent issues.

First of all, all the solutions work on pre-segmented data. Meaning that the solutions process only one parking spot at a time. No solution for automatic segmentation is provided as they propose segmenting the parking lot manually as initial one time only setup. This can be a marginal inconvenience as the

advantages offered may outweigh this issue for most users. However, if implementing a plug and use system is a must, automatic segmentation remains to be solved with these researches. Second issue is the camera angle. PKLot[6] and CNRPark[7] are the datasets used by Roland et al.[8] and Sayuti et al.[9]. Both of these datasets contain images of outdoor lots with an aerial view which covers majority of the parking lot. Hence, generalizing a neural network to work on various camera angles or indoors as well can be a major hurdle which is not tackled in these papers. Finally, in the computer vision based solution proposed by Nazia et al.[1] can be prone to lighting changes and shadows as reported in the paper itself. In case of shadows, the binary color transformation used to separate the car and parking spot may not work as shadows can lead to lower amount of white (used to predict presence of a car in the solution) in the segmented spot leading to poor results.

3.2.1.2 Relationship to the proposed research work

Many advancements have been made in the field of computer vision to solve smart parking problems. Using vision based techniques to solve the problem can lead to higher accuracy but turns in poor generalization. Recent advancements in deep learning have shown that using CNN can give comparable accuracy and can be generalized better compared to traditional approaches. Hence, using a CNN seems to be a better option as of now. However, the generalization of the CNN to work on various camera angles still needs to be addressed.

3.2.2 Optimal Route Planning

More cars means it is harder to find available spots in the parking lot. Even when there's a spot, it's not always easy to get there without dealing with traffic. This can cause traffic jams, confusion, and wasted time and energy. To help with this, we use a smart computer program that finds the best way for you to drive from the entrance to your parking spot, taking into account any obstacles in real-time. This makes things smoother and less frustrating for everyone. This reduces time consumption. Methods used for the optimization of shortest path in route planning are Dijkstra Algorithm and A* Algorithm. Dijkstra's Algorithm explore nodes in order of their distance from the starting node. It maintains a min heap or priority queue to select the node which has a smallest distance. First approach was to find the optimized path using Dijkstra's Algorithm, Impedance function model and the balance unction proposed by Yujin, Ge Xiaoxue.[2].

- The algorithm deals with a graph containing nodes (representing locations) and arcs (connecting paths). If the graph is not weighted, we assume the weight to be infinity.
- The primary goal is to find the shortest path from a source node to the neighboring nodes.

- An Impedance function model is introduced. It emphasizes that the time it takes for vehicles to travel basically depends upon the distance travelled and speed. This model aims to provide effective results. These results are based on user preferences in personalized route planning.
- Impedance function involves using semi-theoretical and semi-empirical functions to determine a theoretical model. It considers the relationship between velocity, traffic volume, and density in the theory of traffic flow.
- It also involves the balance function, which considers the equilibrium between time and distance. It also consider congestion.
- In certain situations, achieving balance between distance and time proves to be the most effective approach in path planning.

The article employs Dijkstra's algorithm to integrate parking lot network map data. This simulates the model using MATLAB for verification. Focus is to plan a route from the starting point to the 35Th point in the parking lot network. Three distinct scenarios are considered for the path. The first uses distance as the weight matrix. The second employs time as the weight matrix. The last incorporates a balance function between distance and time as the weight matrix. These cases aim to assess the algorithm's feasibility and efficiency in navigating parking lots under different conditions, shedding light on its practical applicability.

Optimal factor	Distance length(m)	time consumption (min)
Distance	93	0.6987
time	120	0.3769
Balance	95	0.4253

Figure 3.3: Results proposed by Jujin and G. Xiaoxue[2]

Figure shows the results posted by Jujin and G. Xiaoxue[2] comparing the time taken of different algorithms proposed.

After analyzing the three calculation results, let's compare the first and third cases. In the first case, optimal factor was distance, the path length is 93 meters, and the time is 0.6987 minutes. In third case the optimal factor was balance function, the length is 95 meters, and the time is 0.4253 minutes. Comparing the two, the first path is 2.15% shorter in length, but it takes 39.12% more time. This means that, in terms of efficiency, the third path (using the balance function) is more efficient than the first path. Hence, for the optimal path, balance function is the most efficient.

A* Algorithm is the combination of best first search and Dijkstra's algorithm proposed by Lin Hu, Zhiping Liu.[10] and Vibha S, Sudheer., Shwetha.[11]. It uses both the cost and the heuristic function

to calculate the shortest distance. The process includes,

1. Searching area division network.
2. Adding the starting point, A to the open table.
3. Searching accessible points around the node
4. Deleting the point from the open table and count it in the closed list.
5. Calculate the point $F = G + H$.
6. Calculate whether the point F is the minimum. If no, Adding this point to the closed list. If yes, then node finding is successful. Deleting it from the open table and adding it to the closing table. Is this point being the target point, if no, then go to the 3rd step of this process.

In order to find the optimal path using A* algorithm following steps are followed,

1. Set vehicle start and end positions using RFID reader data (check-in and allotted slot as destination).
2. Use Google Maps to suggest the shortest route, implementing the path-finding algorithm.
3. Compute the best route with the algorithm and check if it has obstacles.
4. If the route is obstacle-free and not congested, follow recommended route.
5. If there are obstacles, algorithm determines the next optimal route.
6. Proceed to drive to the destination endpoint.

A* an efficient path searching algorithm is proposed. MATLAB was used for the testing purpose. A map was given and the system was tested. System passed the tests multiple times. Some advantages are discussed below:

- Combine all path points to smoothen the path, enhancing speed.
- Detect collisions at each distance unit, prioritizing examination before execution.
- Account for both stationary and movable obstacles in path planning.
- Users receive path directions through GPS from a mobile app.
- Implementation of this algorithm in a smart parking system optimizes paths, saving time, fuel, and improving parking efficiency for drivers.

3.2.2.1 Critical analysis of the research item (Strengths and Weaknesses)

Testing revealed certain problems with the system. It's not finished, and by including new features, we can make it better. For instance, a platform for dispatcher management and a warning to avoid crashes during parking could be included. The system will function better with these additions.

When it comes to smart transportation, route planning and road testing is important. It needs to be fitted with a variety of sensors to detect things like the surrounding weather and traffic patterns in the parking lot. There is room for improvement in the existing route-finding process, particularly when there are several possibilities for the same trip. Later on, we can improve the quadratic fuzzy algorithm.

A more thorough investigation is required as the stability of the system has not been thoroughly examined. There is a need for a more in-depth analysis of images from different angles in different conditions. However, the smart parking system can help quickly find parking spots. This will make parking process more efficient. In conclusion, this method has practical value and promising development prospects.

3.2.2.2 Relationship to the proposed research work

As we are providing the user with the shortest path to the parking spot, So the research done is very helpful for us. The Algorithms used for finding the shortest paths are Dijkstra's and A*. As we are working on parking lots, we will have a well-designed heuristic and the graph is reasonably large. A* is generally a more efficient choice for finding the shortest path to a parking spot. It efficiently balances the need for global exploration like Dijkstra's. With this ability to prioritize more promising paths based on heuristic information.

3.2.3 Crime Detection

In this fast-growing world of technology, we see that numerous events happens on daily basis. Based on security concern, there is an alarming need of system to identify such malicious events. Think about a world where smart computers can help keep us safe by watching what's happening around us. They use their special vision to spot trouble and stop crimes before they happen.

Most of the techniques reviewed for crime detection invlove the use of deep learning which needs datasets for its learning process. After thorough research, we've been able to gather XD-Crime[12], and UCF Crime[13] datasets which can be used for any suspicious activity detection. However, this dataset only contain images for outdoor parking lots. Hence, as is the case with parking lot classification, a new indoor dataset may need to be created for better generalization performance.

Talha et. al[3] present a solution to the urgent problem of increasing violent crime in public areas. It explores the diverse factors contributing to violence, including factors like greed, frustration, and social

and economic pressures. The paper suggests the creation of a real-time system for monitoring violent activities using computer vision technology, enabling the rapid identification of violent incidents in public spaces. This system observes people's movements and identifies potential aggressors, promptly alerting law enforcement to prevent violence. It leverages computer vision and neural networks, showcasing both high accuracy and user-friendly design. This paper used the CNN BiLSTM model, which can be used to train and handle our chosen dataset. The accuracy comparison of the model used in paper with other models is given as following in figure.

Methods	Hockey	Movies	Violent Flows
MoIWL [13]	96.8±1.04 %	-	93.19±0.12%
ViF+OVIF [14]	87.5±1.7%	-	88±2.45%
Spatiotemporal Encoder [15]	98.1±0.58 %	100±0%	93.87±2.58%
Conv 3D [16]	98.3±0.81 %	100±0%	97.17±0.95%
CNN-LSTM [6]	97.1±0.55 %	100±0%	94.57±2.34%
CNN-BiLSTM (our model)	94.9%	92.92%	77.31%

Figure 3.4: Talha et al.[3] proposed results

Figures show the comparison between proposed model and its architecture with other existing models and their architecture.

Ijjina et al.[4] introduce an innovative framework for the detection of road accidents through computer vision-based video surveillance. This framework utilizes Mask R-CNN for precise object identification and an efficient centroid-centered object tracking algorithm to analyze surveillance footage. It evaluates the likelihood of accidents based on deviations in speed and trajectory following vehicle interactions. This framework excels in achieving a high Detection Rate and a low False Alarm Rate across diverse road-traffic conditions, including challenging weather conditions. The comparison of the performance of the other models and the model used in this research is given by the author[4] as:

Approach	Diff. Cameras	DR %	FAR %
Vision based model (ARRS) [12]	1	50	0.004
Deep spatio-temporal Model [10]	7	77.5	22.5
Proposed Framework	45	71	0.53

Figure 3.5: Ijjin et al.[4] Reported Accuracy

Comparisons among the performance of other accident detection algorithms with the Algorithm proposed by Ijjin et al.[4].

Another research by H. Ghahremannezhad, H. Shi, and C. Liu[5] focuses on the automated detection of traffic accidents, particularly at urban intersections. It presents a three-step framework that incorporates

YOLOv4 for object recognition, Kalman filter in conjunction with the Hungarian algorithm for object tracking, and analysis of trajectory conflicts for accident detection. This framework adeptly addresses obstacles such as object obstruction, overlapping objects, and changes in object shape. Testing with real traffic video data demonstrates the framework's efficacy in identifying trajectory conflicts and accidents at intersections with a minimal rate of false alarms and a high rate of accurate detection, even under varying lighting conditions. The comparison of the collision detection with other methods given by author is:

Methods	Num. of videos	DR %	FAR %
Ki et al. [26]	1	63	6
Singh et al. [27]	7	77.5	22.5
Ijjina et al. [28]	45	71	0.53
Wang et al. [29]	–	92.5	7.5
Pawar et al. [30]	7	79	20.5
Proposed method	29	93.1	6.89

Figure 3.6: Ghahremannezhad et al.[5] Proposed Results

Performance comparison of method by Ghahremannezhad et al.[5] with other representative accident detection methods.

To summarize, these papers highlight inventive approaches for addressing critical issues like the rise in violence and road accidents. They harness the capabilities of computer vision and object tracking technologies to create real-time monitoring and detection systems that offer both precision and adaptability in diverse and challenging scenarios.

3.2.3.1 Critical analysis of the research item (Strengths and Weaknesses)

While the solutions proposed in the research papers promise a high degree of accuracy, they are not without their issues.

Firstly, the solution proposed by Raihan Talha, Bandapadya and Monirujjaman Khan[3] has shown impressive performance, but it's important to validate it further using established datasets that allow for the identification of multiple violent acts involving one or many individuals. Also, it does not identify if the person is holding any weapon or not. It needed to check further that if any un-authorized person is holding and illegal weapon or not. Secondly, the solution proposed by Ijjina et al.[4] has high accident detection rate i.e 73%. But it has some issues that includes the failure of system due to high traffic congestion which may result in false detection. Moreover, it uses wide range cameras with aerial view. Any obstacle in the view of field may result in failure as the system would not be able to calculate the trajectory of the targeted vehicle. Moreover, it uses the aerial view that can be used in the wide areas to cover more area. But these proposed solutions can be more costly if we have to install them in area where the aerial view cannot be effective.

3.2.3.2 Relationship to the proposed research work

The studies we have reviewed cover a wide range of scenarios relevant to crime detection, including parking lots where collisions and criminal activities may occur. These insights can aid in the enhancement of our model's reliability, efficiency, and security. By identifying potential issues and triggering an alarm, our model can serve as a deterrent to criminal activities. It can alert individuals to avoid certain areas or notify relevant authorities to intervene promptly and accurately. This proactive approach not only ensures the safety of individuals but also contributes to the overall security of the area.

CNN BiLSTM model, can be used to train and handle our chosen dataset. Ijjina et al.[4] introduces an innovative framework for the detection of road accidents through computer vision-based video surveillance. This framework utilizes Mask R-CNN for precise object identification and an efficient centroid-centered object tracking algorithm to analyze surveillance footage. It evaluates the likelihood of accidents based on deviations in speed and trajectory following vehicle interactions. Also, Ghahremanezhad et al.[5] focuses on the automated detection of traffic accidents, particularly at urban intersections. It presents a three-step framework that incorporates YOLOv4 for object recognition, Kalman filter in conjunction with the Hungarian algorithm for object tracking, and analysis of trajectory conflicts for accident detection.

3.2.4 Relevant Applications

This section explores several applications that offer functionalities similar to those of our project, albeit partially. Each application is briefly described, highlighting its unique features and limitations, particularly where they diverge from the capabilities of our project.

These applications serve as benchmarks for our project, allowing us to understand the current landscape and identify areas where our project can offer improved or additional functionalities. By understanding the strengths and weaknesses of existing solutions, we can better position our project to address unmet needs and provide a more comprehensive and effective solution.

Though these applications offer valuable insights, our project aims to go beyond by integrating these functionalities into a cohesive, user-friendly, and efficient system. Our goal is not just to replicate but to innovate, ensuring that our project delivers a superior user experience and meets the evolving needs of its users.

3.2.4.1 Sparkle

Sparkle[14] is a project hosted on GitHub which uses computer vision and deep learning to classify parking lot. The system works by utilizing CNN models including CNN, R-CNN and YOLO. The system takes an image of a parking lot as input and as output gives all the occupied and available parking

lots.

Compared to our proposed solution, the system does not provide a way to disseminate parking information to drivers. Without informing the driver, such an application fails to serve its purpose. Furthermore, there is no module keeping track of the cars entering and leaving the lot which our solution solves by keeping track of car licenses. This system also works on images so a separate module has to be created to feed live video in the form of frames to this system.

3.2.4.2 CVedia

Utilizing computer vision and deep learning algorithms, CVEDIA[15] smart parking solutions offer a way to monitor parking spaces in real-time using pre-installed cameras, eliminating the need for expensive sensor-based methods. This system is developed using computer vision technique with the help of pre-installed camera. That captures the image of parking lot and process it for empty slot. These systems accurately detect and categorize vehicles parked in designated slots, even in challenging weather conditions.

This approach presents an efficient and cost-effective means of managing parking lots, especially in large-scale settings. Although this system has much accuracy, and it is designed to facilitate the users/drivers. But it does not serve the whole purpose to facilitate the users as it do not show the path to the desired or nearest parking spot available. So the challenge remains there to facilitate to user to park their car without any hustle.

3.2.4.3 AR Based Parking Navigation

AR Based Parking Navigation[16] is a project available on GitHub. Enhancing Urban Mobility via AR-based Parking Navigation System. Images are captured from an overhead CCTV camera. The system works by utilizing CNN models and YOLO. Determines the optimal route to the closest available parking spot. Relies on the Mapbox Navigation SDK, which offers a range of features including route calculation and turn-by-turn instructions.

The experiments conducted to evaluate the proposed system have provided valuable insights. While the system generally performs well in accurately placing visual cues and providing interactive driving guidance, there are challenges related to reliability and stability, particularly in localization and object placement. The limitations of VPS technology in highways and roads, as well as high-speed driving scenarios, also highlight the need for further improvement.

3.3 Literature Review Summary Table

Throughout the research, we've studied various research papers to get an in-depth knowledge of the work already done in the field. The results are represented in the following table which shows the authors name, relevance to our project, method used, dataset used, and the reported results.

Table 3.1: Review of the Research Papers

This summary Table contains detailed analysis of studies relevant to parking space classification, crime detection and parking lot routing using computer vision.

Authors	Relevance to Project	Method Used	Dataset	Results
Nazia et al.[1]	Parking spot classification	Image processing and Computer Vision	None	Average Accuracy: 99.5%
Roland et al.[8]	Parking spot classification	CNN using Tensor-flow Lite	PKLot[6], CNRPark-ext[7]	Accuracy: 93.79% F1 Score: 94.21% Time Per Image: 0.219s
Sayuti et al.[9]	Parking spot classification	Modified mAlexNet (CNN)	CNRPark-ext[7]	Average Accuracy: 99.12%
Yujin, Geng Xiaoxue[2]	Optimal Route Planning	Dijkstras Algorithm	CARPK[17]	Balance function as the optimal path planning is most efficient
Hu, Lin, Liu, Zhiping[10]	Optimal Route Planning	Heuristic Solution (A*)	None	Shortest effective path is obtained by testing on MATLAB
Sylvia Vibha, Sudheer Shetty, J ShwethaR.[11]	Optimal Route Planning	Heuristic Solution (A*)	PKLot, CNRPark-EXT[18]	Shortest effective path is obtained if no obstacle
Raihan Talha, Bandyapadya, Moniruj-jaman Khan.[3]	Crime Detection	CNN-BiLSTM	Hockey Fight[19]	Average Accuracy: 80%
Paul Ijjin.[4]	Crime Detection	Mask R-CNN	Youtube Videos	Accuracy: 88.37%
Ghahremannezhad, Hang Shi, Chengjun Liu.[5]	Crime Detection	YOLOv4	Accident Detection[20]	Accuracy: 93.10%

The following table provides a summary of the relevant applications we have researched. It highlights their key features as well as their limitations.

Table 3.2: Summary Table of Similar Applications

This summary Table details the relevant applications researched, highlighting their key features and limitations where they differ from our proposed project.

Application	Features	Limitations
Sparkle[14]	Parking spot classification	Missing features Routing, tracking entered cars
AR Based Parking Navigation[16]	Parking spots detection, Routing	Missing features tracking entered cars
CVEDIA[15]	Parking spots detection	No routing and tracking cars

3.4 Conclusion

The research conducted on spot classification, optimal route finding and anomaly detection in parking lots using computer vision showcases potential for an innovative solution as proposed in this paper which can combine all these features in a single platform to alleviate the issues faced by drivers all around today. While all the researches demonstrate remarkably high accuracy and efficiency, they face some common issues and limitations namely manual setup for segmentation, varying environments and susceptibility to lighting and weather changes.

By taking these studies as a starting off point, we aim to leverage their advantages and hopefully work towards reducing their disadvantages. The current need is to devise generalized models, automation segmentation and improving its performance in varying environments.

Chapter 4 Software Requirement Specifications

This chapter provides a comprehensive overview of the project's key features. It delves into both the functional and non-functional requirements that the project necessitates. Furthermore, it outlines the design of the database and conducts a risk analysis associated with the project.

4.1 List of Features

Major features of our system are as follows.

- Classify all the parking spots in the lot.
- Store information related to entry and exit of vehicles in the parking lot.
- Detect and notify about any risks or anomalies like accidents, fires, etc in the lot.
- Disseminate parking spot availability information to drivers for decision making.
- Create a map of the parking lot for easy visualization.
- Calculate and display shortest route to closest available parking spot.

4.2 Functional Requirements

This section highlights the functional requirements of our system which relate to its technical functionalities. These requirements include:

- The system should allow the admin to setup the system any time by selecting cameras and highlighting parking spots.
- The system should allow the admin to create a map of the parking lot for visualization purposes.
- The system should classify all the parking spots highlighted by the administrator.
- The system should allow the admin to see a record of the vehicles entering and exiting the system.
- The system should display information related to parking spot availability on a central display for drivers.
- The system should allow the admin to view the feed of all CCTV cameras.
- The system should display parking spot information to admin for easier management.
- The system should store information related to entry and exit of vehicles.
- The system should trigger an alarm on the admin side to inform about any anomalous activity.

- The system should display the footage of crime or accident.
- The system should display the confidence level of the accident, indicating the degree to which the system is certain about the incident.
- The system should display an interactive map with highlighted optimal route and available parking spaces.

4.3 Quality Attributes

Quality attributes are integral to the performance evaluation of a software system. They provide a checklist that aids in assessing the quality of the software. The quality attributes of our system are:

- The system shall be easy and intuitive to use.
- The system should perform its functionalities with minimal errors and high accuracy.
- The system shall be easy to modify and upgrade utilizing modules which will be easy to maintain and modify.
- The system shall be available 24/7 having minimal down time.

4.4 Non-Functional Requirements

This section describes all the non-functional requirements of our system, focusing on factors like performance (time, accuracy, errors), extensibility, etc.

- The system shall be able to classify parking spots in real-time (in less time than time taken by the driver to reach the entrance of the lot ideally under 5 seconds).
- The system shall classify at least 50 parking spots in under 1 second.
- The system shall display only the current parking spot information to drivers minimizing parking errors.
- Access controls should be implemented to restrict access to sensitive data ensuring privacy.
- The system should be efficient enough to inform the admin side about anomalous activity timely in order to minimize the impact.
- The system shall display the shortest route to driver in real time. Ideally it should be shown to driver before he reaches the entrance of parking lot under 5 seconds.

4.5 Assumptions

Following assumptions will be made when developing our system.

- Only admins will be allowed to use the system/server used to manage the parking lot software.
- Parking lot premises will be covered by CCTV cameras and there is at least 1 outdoor camera at entrance for scanning license plates.
- Lighting conditions are sufficient enough that the camera can easily make out all the vehicles and parking spots.
- An admin is monitoring the system to respond to any security alert generated by the system.

4.6 Use Cases

The is section presents use cases of some functionalities offered by our system.

4.6.1 System Setup

Following use case highlights the steps followed to setup the system.

Name		System Setup	
Actors		Admin	
Summary		The user will select multiples outdoor and indoor cameras whose feed will be used by our system for choosing parking spots.	
Pre-Conditions		System must be deployed at the server. All the cameras must be in working condition and their integration must be in place with our system.	
Post-Conditions		On system startup, parking spots will be shown to the user.	
Special Requirements		None	
Basic Flow			
Actor Action		System Response	
1	The user opens the system setup page.	2	The system setup page is displayed asking for outdoor camera.
3	The user selects valid working outdoor camera.	4	The system displays a page asking for indoor cameras.
5	The user selects valid working indoor cameras.	6	The system displays a separate page for parking spot segmentation.
7	The user selects all valid parking spots and clicks finish.	8	The system saves the information and starts the system.
Alternative Flow			
9	The user select invalid camera.	9-A	The system responds with an error message: Invalid Camera.
10	The user select invalid parking spot.	10-A	The system responds with an error message: Invalid Parking Spot.

4.6.2 System Start

Following use case highlights the steps followed to start the system.

Name		Initiate System	
Actors		Admin	
Summary		The admin initiate the system. The system will show multiple options i.e camera feed, slots information, vehicle information and security alerts.	
Pre-Conditions		System setup should be done before in order to start the system.	
Post-Conditions		The main display screen for drivers will be in working condition i.e showing slots either empty or occupied and optimal route.	
Special Requirements		None	
Basic Flow			
Actor Action		System Response	
1	The user clicks the start system button.	2	The system main screen is opened showing camera feed, security alert, slots and vehicle information
Alternative Flow			
3	The user clicks start system button	3-A	The system responds with an error message: System Setup not completed.

4.6.3 Security Alerts

Following use case highlights how the system notifies admin about anomalous activities.

Name	Security Alerts		
Actors	Admin, System		
Summary	The system will give the alert to admin about any criminal activity and admin will response to the event on time.		
Pre-Conditions	There will a person from admin who will be monitoring the system real-time.		
Post-Conditions	The system will give the security alert of activity, after confirming that admin will respond to that.		
Special Requirements	None		
Basic Flow			
Actor Action		System Response	
1	The admin opens the security alerts page.	2	The system will show the footage of activity detected
3	The admin will confirm the type and accuracy for the activity.	4	The system will calculate and show the results of type and accuracy percentage.
Alternative Flow			
5	The admin confirms that it is a false alarm.	5-A	The system showing the less confidence level percentage will discard the alter

4.6.4 Vehicle Information

Following use case highlights how the admin can view vehicle records information including entry/exit time and license number.

Name	Vehicle Information		
Actors	Admin		
Summary	The admin will be able the see the records of vehicles stored in database.		
Pre-Conditions	The system setup has been done already.		
Post-Conditions	None		
Special Requirements	None		
Basic Flow			
Actor Action		System Response	
1	The admin opens the vehicle information page.	2	The system will display the information of vehicle including the details of license plate and their entry and exit time.
Alternative Flow			
1	The admin opens the vehicle information page.	2-A	There is no vehicle record in the system. The system will display an message.

4.6.5 Camera Feed

Following use case highlights the steps followed by the admin to view camera feed of the lot.

Name		Camera Feed	
Actors		Admin	
Summary		The admin will be able to see the feed of different cameras.	
Pre-Conditions		The system setup has been done already.	
Post-Conditions		None	
Special Requirements		None	
Basic Flow			
Actor Action		System Response	
1	The admin opens the camera feed page.	2	The system will display the camera feed of different cameras installed at different locations.
Alternative Flow			
1	The admin opens the vehicle information page.	2-A	There is no camera feed from any camera due to any reason like electricity shut down.

4.7 Hardware and Software Requirements

All the hardware and software requirements that will be required to develop and deploy the project are mentioned in this section.

4.7.1 Hardware Requirements

This section mentions hardware needed for running the system properly. These requirements are:

- A modern computer acting as a central server to control the system.
- Sufficient storage to store vehicle records.
- CCTV cameras covering all regions of interests (parking spots and entrance) of the parking lot.
- A GPU to aid faster processing of CNN and other deep learning models.
- A networking switch/router to connect all the cameras in the premises to the central server.
- Lan cables for hardware connectivity.

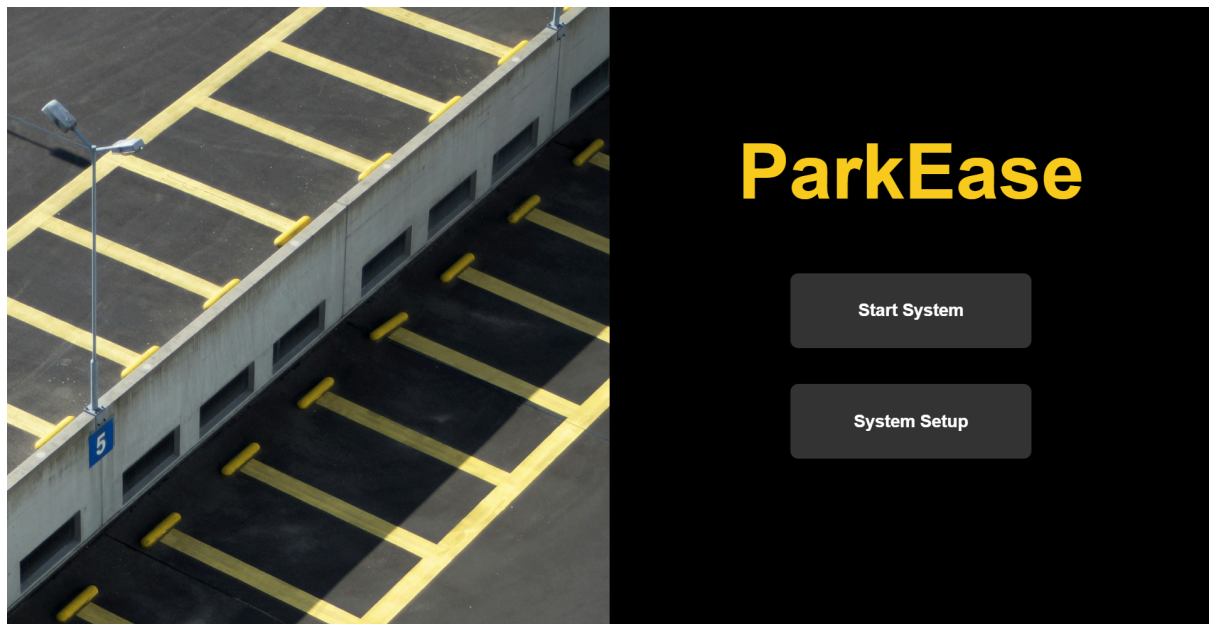
4.7.2 Software Requirements

This section of the report highlights some of the software requirements for smooth development of the system. These include:

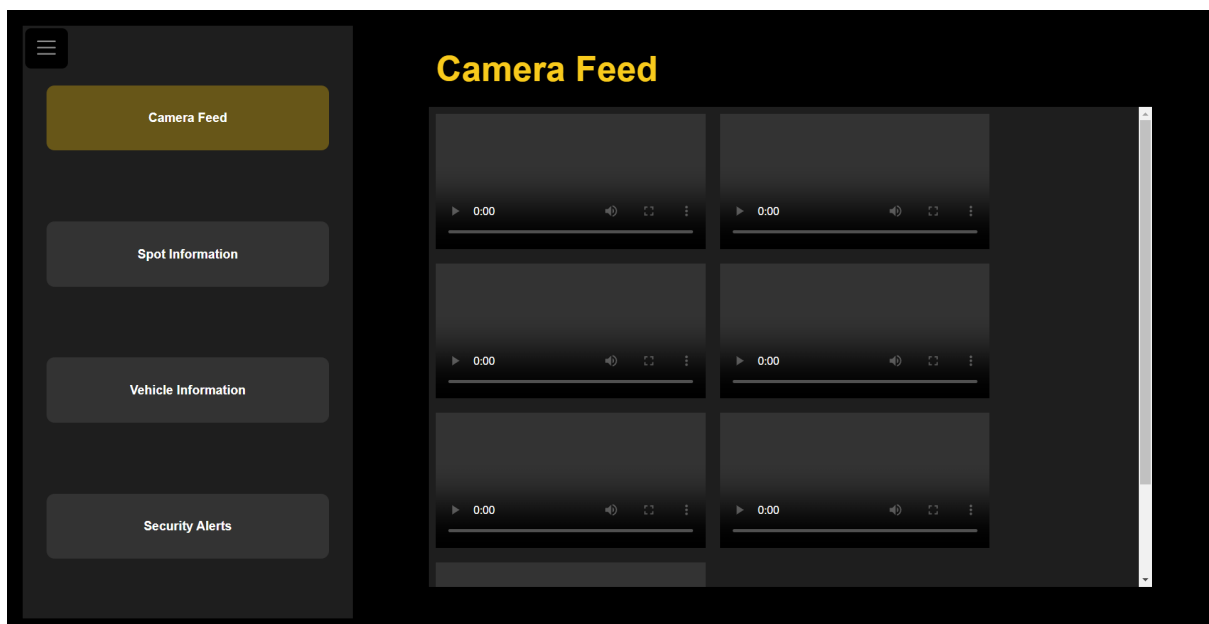
- Jupyter Notebook/ PyCharm
- Python libraries like OpenCV, TensorFlow, EEL
- Microsoft SQL Server
- Draw.io

4.8 Graphical User Interface

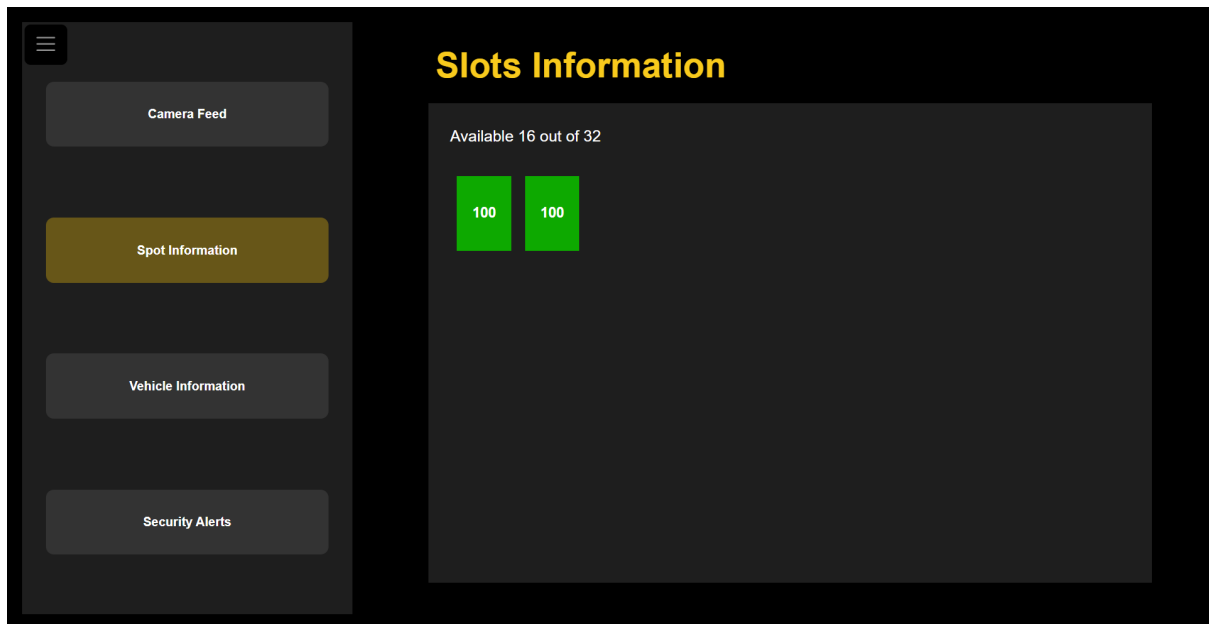
The GUI dumps of each screen, with reference to the functionality allowed and intended user are given in this section.

**Figure 4.1: Start Screen**

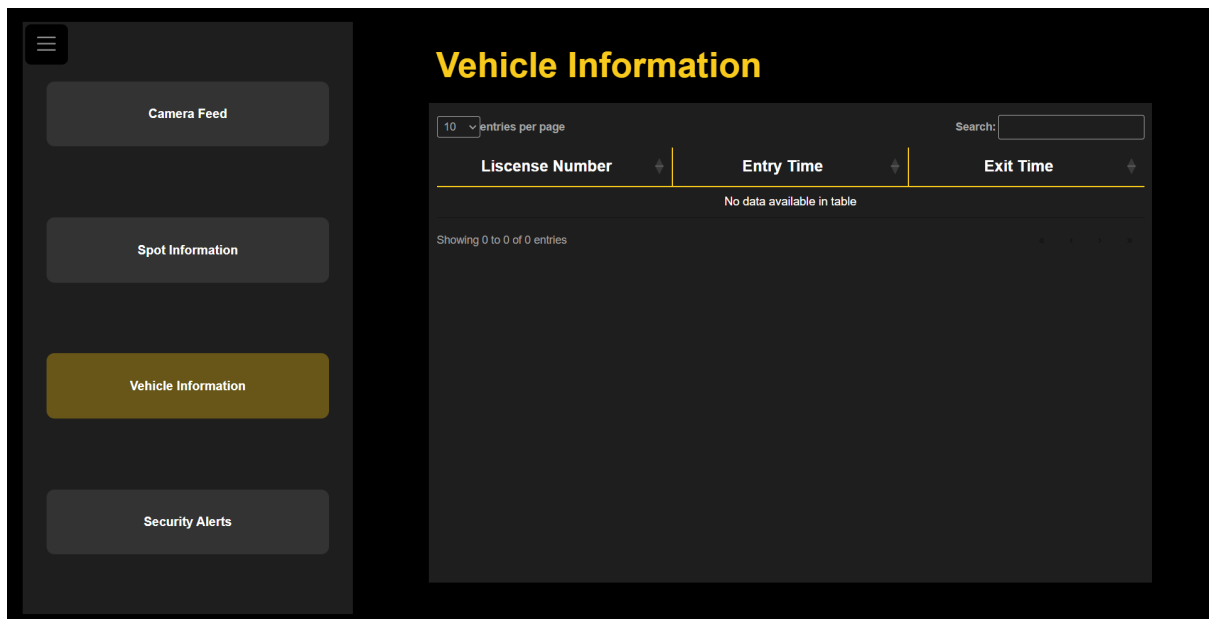
Admins can choose to setup the system or start with existing configurations.

**Figure 4.2: Live CCTV Feed Screen**

Admins can monitor the premises with the footage from all cameras in the lot.

**Figure 4.3: Parking Slots Information Screen**

Admins can easily monitor all the slots in the lot.

**Figure 4.4: Vehicle Record Screen**

Admins can see record of all the vehicles that have used their lots by checking their license numbers and entry/exit times.

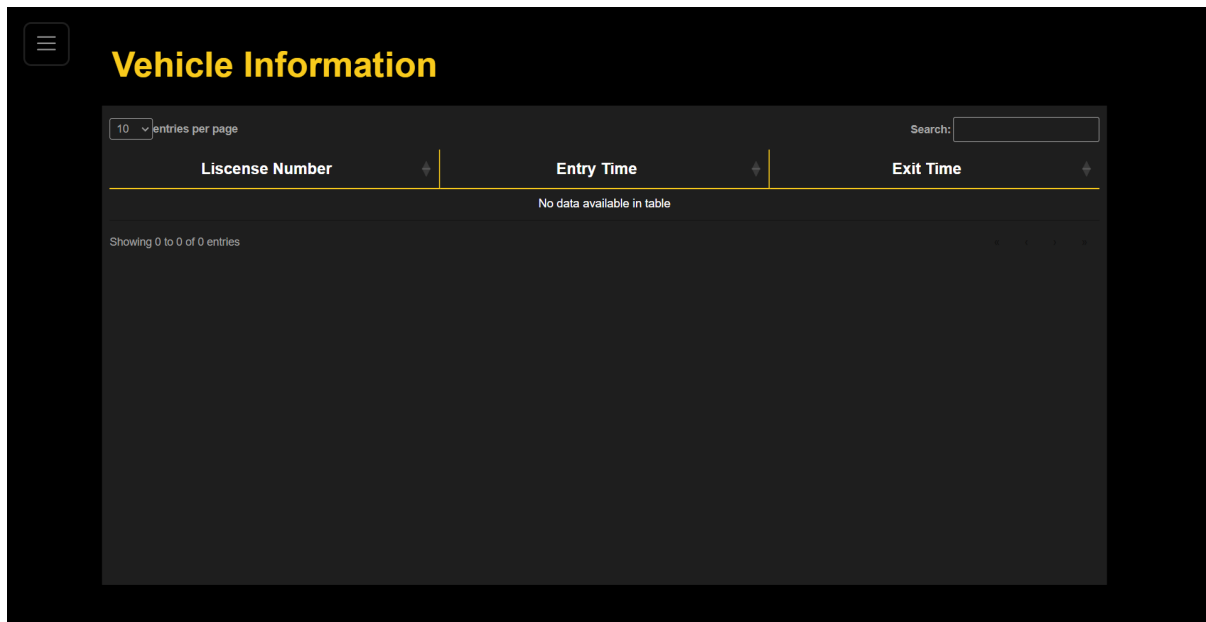


Figure 4.5: Vehicle Record Screen - Collapsed NavBar

Shows the functionality of collapsible side navigation bar.

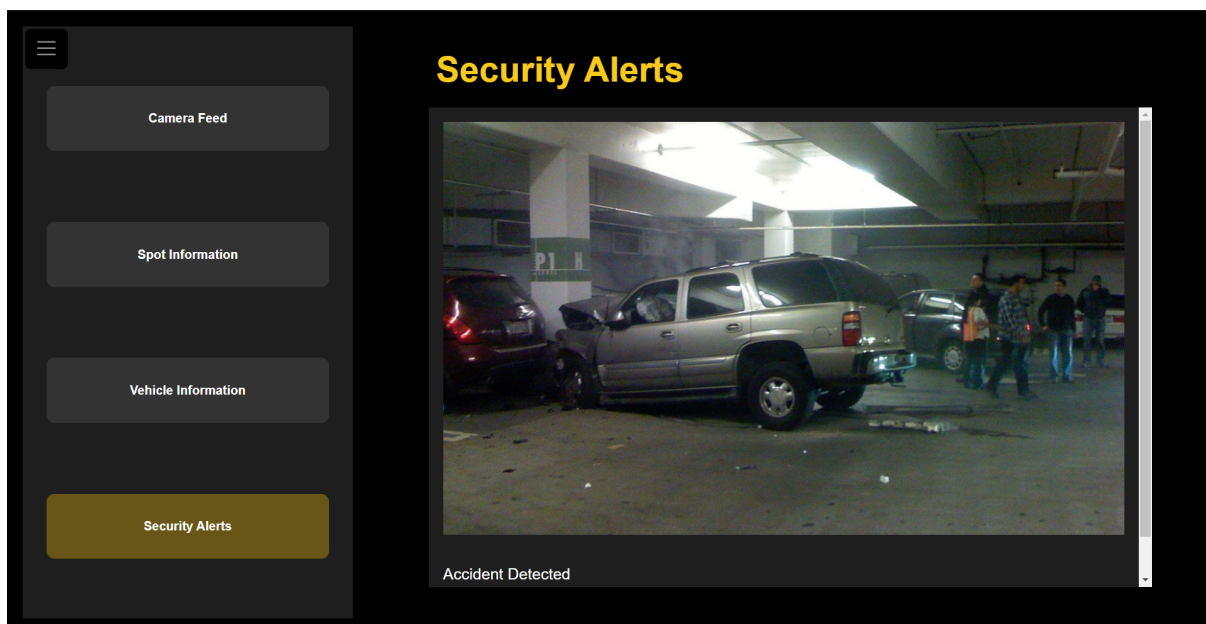
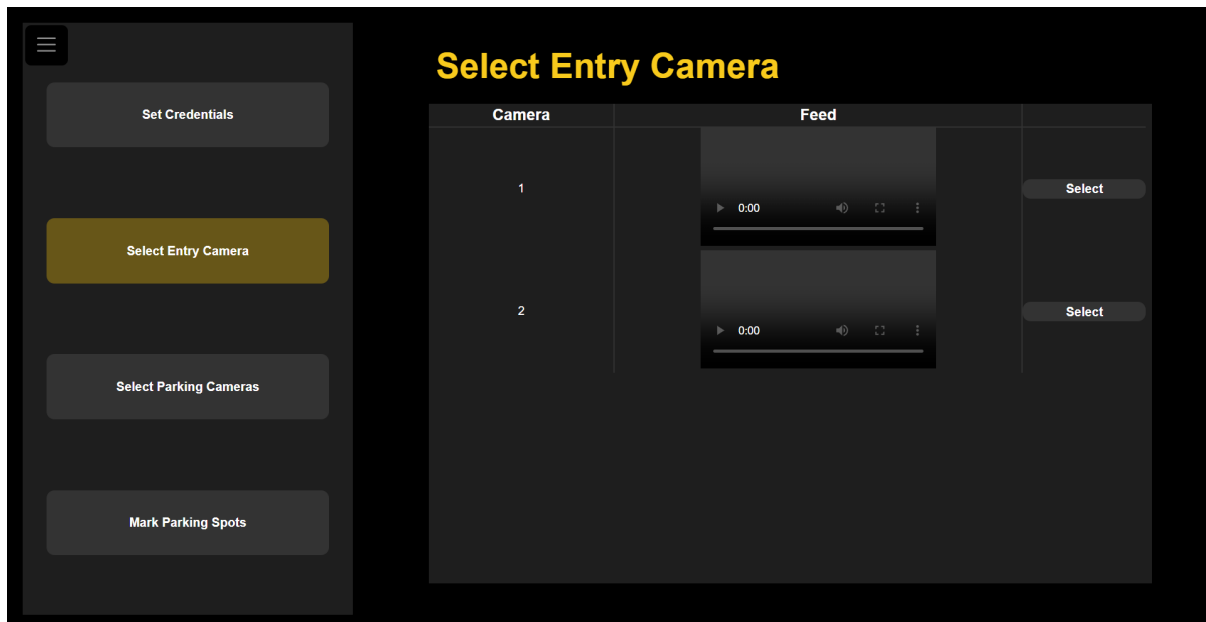
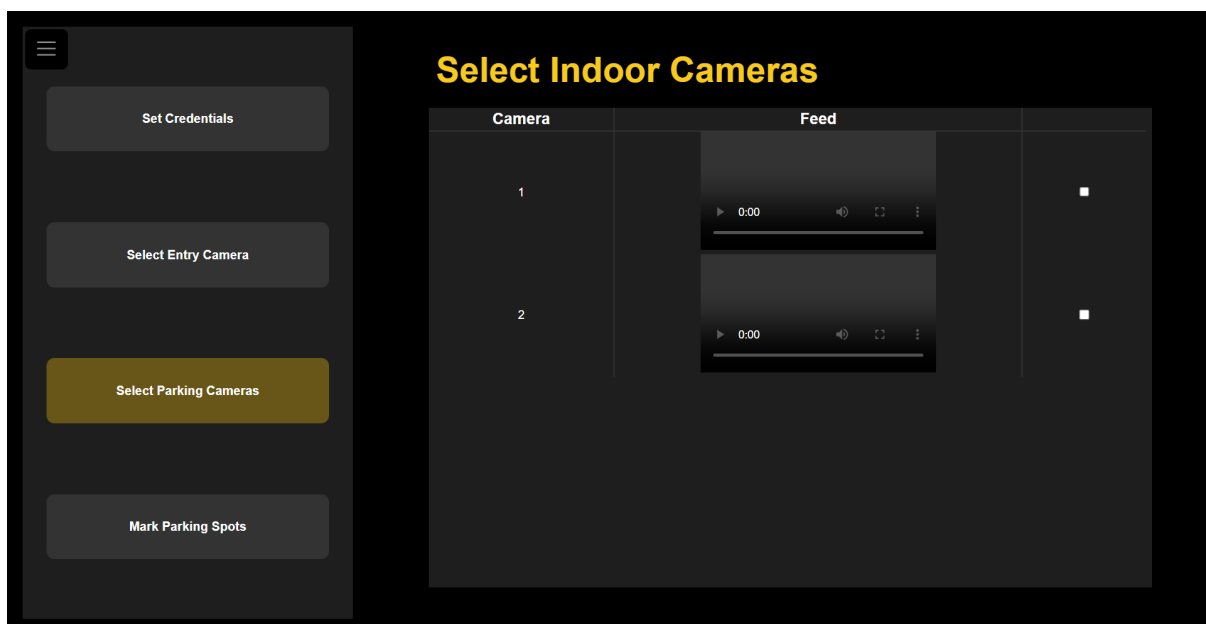


Figure 4.6: Security Alert Screen

Admins can see the type of security alert, its location with live feed and the accuracy for informed decision making.

**Figure 4.7: Outdoor Camera Selection Screen**

Admins can start the setup by selecting the outdoor camera to be used for license plate scanning.

**Figure 4.8: Indoor Camera Selection Screen**

Admins can continue the setup by selecting all the indoor cameras from list of cameras detected.

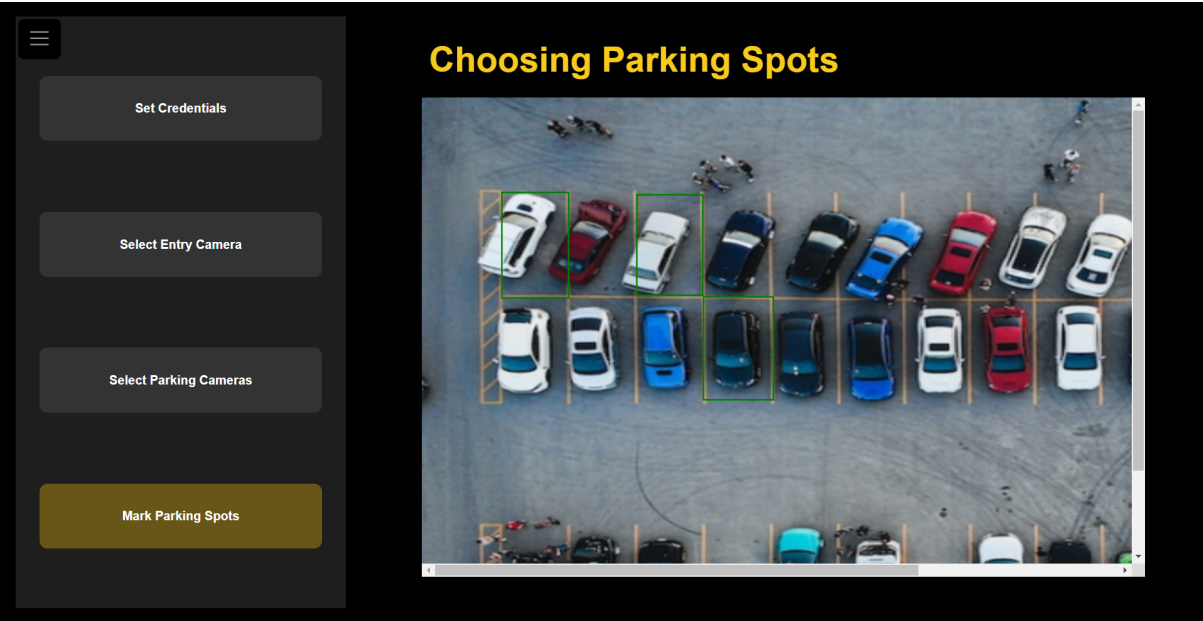


Figure 4.9: Segmentation Screen
Admins can finish the setup by selecting parking spots by using a pointing device like a mouse.

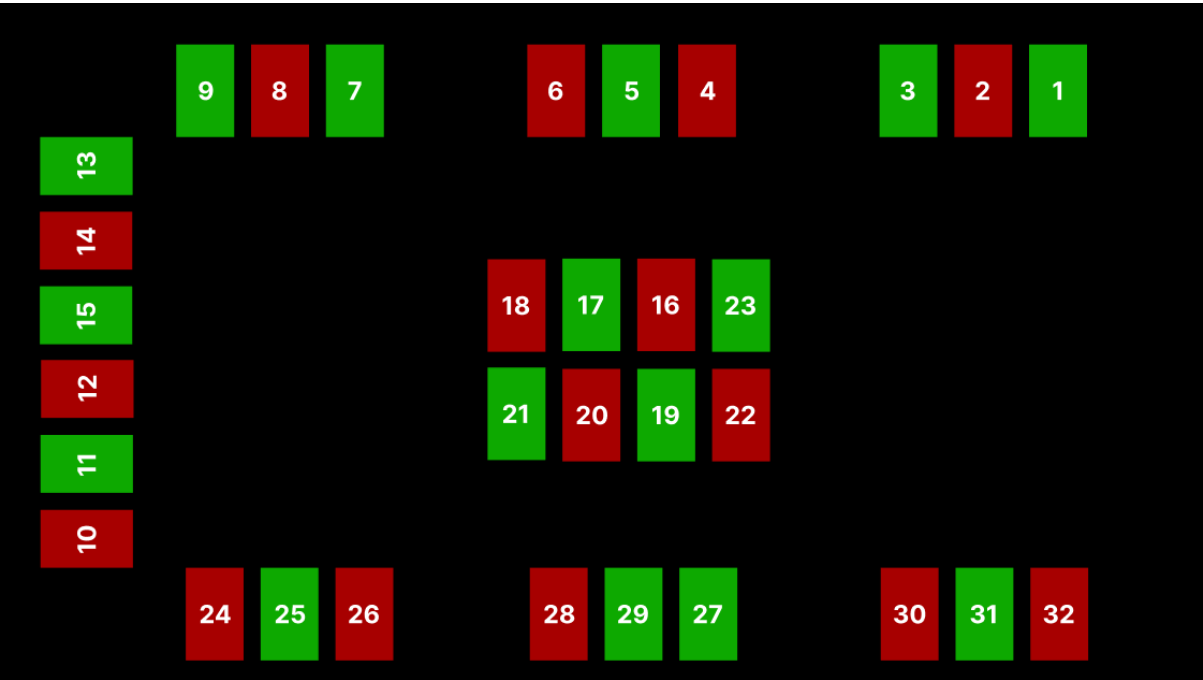


Figure 4.10: Driver Information Screen
Drivers can utilize the map created to see available spots for decision making.

4.9 Database Design

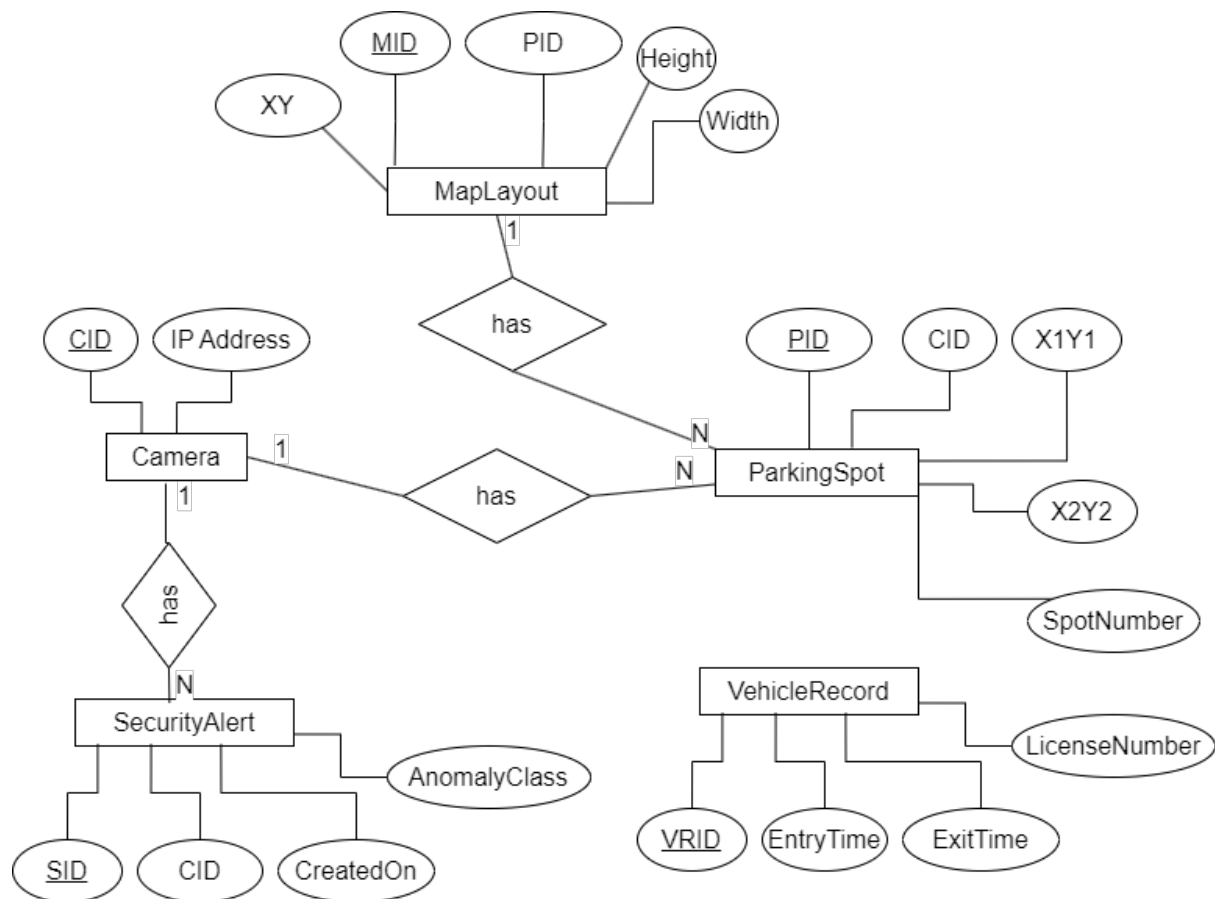
This section highlights the database design of our system by providing an ER diagram and a data dictionary describing the entities and attributes.

4.9.1 ER Diagram

The Entity-Relationship (ER) diagram provides a detailed representation of the database design for our system. It encompasses various entities, each of which plays a crucial role in the system's functionality. These entities include:

- **Camera:** This entity is responsible for storing information related to cameras installed in the lot.
- **ParkingSpot:** This entity stores the individual parking spaces within the lot.
- **VehicleRecord:** This entity maintains a record of all vehicles that enter and exit the parking lot.
- **MapLayout:** This entity stores a layout of the parking lot, aiding in navigation and spot identification.
- **SecurityAlert:** This entity is responsible for storing alerts in case of any security breaches or unusual activities.

Each of these entities interacts with others, forming a complex network that enables efficient operation of the system. The ER diagram serves as a roadmap for understanding these interactions and the overall structure of the database.

**Figure 4.11: ER Diagram**

This ER diagram describes the database design of our system.

4.9.2 Data Dictionary

Data dictionary given below defines all the entities, their attributes and data-types.

Table 4.1: Data Dictionary Table

This table gives details of all the database entities and their attributes.

Entity	Attribute	Datatype	Relation To	Nullable	Description
Camera	CID	UniqueIdentifier		No	Primary Key of Cameras table
	IPAddress	String		No	Network address used to access the camera
ParkingSpot	PID	UniqueIdentifier		No	Primary Key of ParkingSpot table
	CID	UniqueIdentifier	Cameras	No	Primary key of Cameras table inserted as a Foreign Key
	X1Y1	String		No	Coordinate of top left corner of a parking spot
	X2Y2	String		No	Coordinates of bottom right corner of the parking spot
	SpotNumber	Integer		No	A number assigned to each spot
VehicleRecord	VRID	UniqueIdentifier		No	Primary Key of vehicle records table
	License Number	String		No	License number of the vehicle entering the lot
	Entry Time	DateTime		No	Entry time and date of vehicle
	Exit Time	DateTime		Yes	Exit time and date of vehicle

4.10 Risk Analysis

Two major risks are associated with this project which are Time and Performance constraints.

Project involves various phases like data collection, including public datasets and custom dataset, data pre-processing, model training, testing, model implementation, and front-end implementation. Given the amount of work, time is a major constraint that needs to be managed during this project. Next in performance. As this project is to replace ultrasonic sensors which have little to no error rate, our

system has to perform somewhat comparable to traditional solutions to be effective. Thus, performance constraints also pose a major risk.

4.11 Conclusion

To conclude, this chapter highlights major features of our system like crime detection, parking lot classification, driver routing. It also defines some functional and non-functional requirements, provides our GUI design in the form of images, and finally describes the database design of our system and assesses some associated risks.

Chapter 5 Proposed Approach and Methodology

This chapter presents the proposed approach and methodology for the development of the system under discussion. It encompasses a comprehensive discussion on parking lot classification, crime detection, and driver routing. For each relevant section, the techniques employed for data collection and pre-processing are also elaborated upon. This systematic approach ensures a thorough understanding of the system's design and functionality, paving the way for its successful implementation.

5.1 Parking Lot Classification

This section of the methodology chapter is concerned with the implementation details of our parking lot classification module which will be used to classify the parking spots as either available or occupied.

5.1.1 Data Collection

We have selected two datasets which are available publicly. These are PKLot[6] and CNRPark-EXT[7]. The PKLot dataset has around 12,400 images of entire lots and 690,000 images of individual spots, taken in various weather and lighting conditions. CNRPark has roughly 150,000 images collected from 164 different parking lots.

5.1.1.1 Segmentation

The first operation is segmentation where the pictures of entire lot will be taken and individual spaces will be segmented out which will go through further pre-processing stages.

5.1.1.2 Resize

Segmented images will then undergo an image resizing stage where all the images will be scaled to a predefined set size of 150x150x3 to remove size variance among images from different datasets.

5.1.1.3 Labeling

Finally, all the processed images will be labelled as either occupied or empty resulting in a final completed dataset which will be used for training and testing the model.

5.1.2 Model Details

The model which will be used is a CNN as proposed by Sayuti et al.[9] which uses 3 convolutional layers and two fully connected layer. The model uses a kernel size of 13x13 for the first layer, 5x5 for

the second layer and 3x3 for the last convolutional layer. The main reason for selecting this model is that it offers a reasonable trade-off between accuracy and processing time allowing us to create a real-time classifier with quite accurate performance. TensorFlow framework will be used for implementation, training and testing the model.

5.2 Optimal Route Planning

In this section, we will be implementing optimal route for the drivers to easily go to their spot without any hindrance. They only need to follow the route to go to their desired spot which will result in low stress and less time consumption.

5.2.1 Techniques

The techniques we will be using for optimal route are Dijkstra's and A-star algorithms. Dijkstra's algorithm guarantees the optimality of the solution as it always finds the shortest path. A-star uses a heuristic function that provides an estimate cost from current node to the goal. We can adjust the heuristic function according to the situation.

5.3 Crime Detection

In this methodology chapter, we will be implementing a system designed to inform the relevant authorities promptly, thereby ensuring the timely execution of security measures.

5.3.1 Data Collection

The dataset we will use is UCF Crime Dataset[13]. This data set has the total image count for the train subset 1,266,345 and 111,308 image count for test subset. This dataset has various different classes which include abuse, road accidents, fighting, assault etc.

5.3.2 Data Pre-Processing

The dataset undergoes a series of preprocessing steps. Initially, gray-level transformation is applied to the data, followed by resizing the images to a uniform 50x50 dimensions. To comply with the requirements of the image data generator, the reshaping process transforms the images into a 4D matrix. Additionally, the final step involves labeling each image, ensuring comprehensive and organized categorization for subsequent stages of analysis.

5.3.3 Model Details

The model we will be using for this purpose is CNN BiLSTM. CNN is used to process each frame from video and extract information and LSTM will be used to detect any sequential flow and then the information will be compared to previous one. We have opted for this model due to its exceptional accuracy.

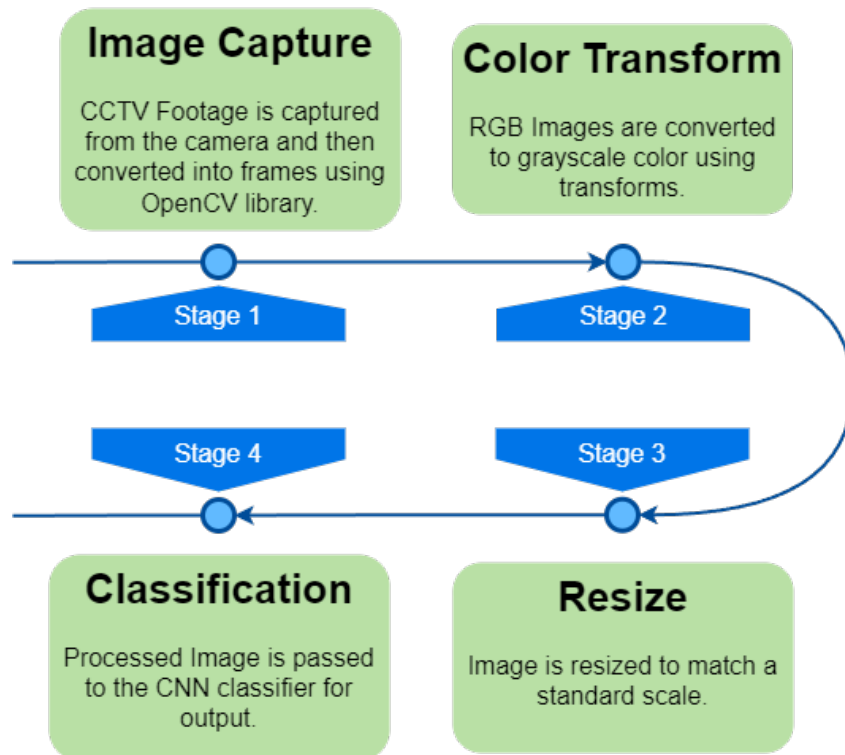


Figure 5.1: Flow Diagram

Shows a overall flow of the Crime Detection and Parking classification module.

Chapter 6 High-Level and Low-Level Design

This chapter offers a comprehensive design overview of our system, encompassing both high-level and low-level perspectives. It includes the presentation of the architecture diagram, sequence diagrams, and class diagrams, each of which contributes to a thorough understanding of the system's structure and functionality.

Moreover, this chapter provides a succinct system overview, shedding light on the overall operation and purpose of the system. It also delves into the design considerations that have been taken into account during the development process, ensuring that the system is robust, efficient, and user-friendly.

Additionally, the strategies and policies that guide the system's operation and evolution are discussed. These provide a framework for decision-making and problem-solving within the system, promoting consistency and effectiveness in its operation.

Together, these elements offer a holistic view of the system's design, providing all the necessary information for understanding its operation, maintenance, and potential for future development.

6.1 System Overview

Through the integration of security cameras and computer vision techniques, our application that is to be developed makes the parking process feasible and easy for drivers. Our system software has the following modules.

6.1.1 Parking Spot Classification

In this phase our system will use computer vision algorithms to identify and classify parking spot in the image as either available or occupied. User will be able to see the map on the main display screen. The map will show the available and occupied slots. Thus, it will allow the user to take decision accordingly.

6.1.2 Optimal Routing

This phase deals with the optimal route planning. By using shortest path algorithms, our system will show the user the optimal route on the main display screen. This will reduce the time consumption and stress involved in finding the empty spot.

6.1.3 Crime Detection

By using different models and algorithms, our system will detect unusual activities in the parking plaza. An alarm can be configured to notify security personnel about this usual activity.

6.1.4 License Plate Recognition

By using OCR model, we can detect license plates. This feature will enhance security of the parking plaza. As this information will further allow us to record the entry and exit time of each vehicle.

6.2 Design Considerations

For our application to be successful, we need to cater to every problem and dependency. So our end user does not face any problem while using the application. This section describes many of the issues that need to be addressed before attempting to devise a complete design solution.

6.2.1 Assumptions and Dependencies

Following are the assumptions and dependencies are associated with our application that should be addressed for successful result.

6.2.2 Software Requirements

- Python libraries such as Open CV, TensorFlow should be available.
- Software such as Pycharm, Jupyter Notebook, Google Colab, Microsoft SQL Server should be available.

6.2.3 Hardware Requirements

- User will have a modern computer acting as a central server.
- User will have a adequate storage for saving vehicle records.
- Parking plaza will have cameras covering all the spots and the entrance.
- A GPU to enhance the speed of processing for deep leaning models.
- LAN cables should be available for the connectivity purpose.

6.2.4 General Constraints

Our project aims to provide easy to use platform for the management. However, there are certain limitations or constraints that can hinder our system performance. Such constraints should be kept in mind to give the best possible output for our users.

- CCTV cameras not working.

- Parking plaza contain spots which are not covered by the cameras.
- Lighting condition not suitable for our system.
- Computer and storage device not optimal for the system.
- Unavailability of software such as Jupyter Notebook, SQL, Pycharm, Google colab, Python libraries for the optimal result.
- Smooth connection between the cameras and our system.
- User vehicle information should be kept secure to avoid any dispute.
- The system should be fast enough to update the map on main screen before vehicles reach the entrance.
- The system should show shortest path which avoid any type of hindrance.
- The system should update the security personals in case of any violence in parking lot.

6.2.5 Goals and Guidelines

Following are the goals and guidelines which dominates the design of our system's software.

- User-Friendly Interface
- Efficient Parking Spot Management
- Scalability and Flexibility
- Security and Privacy
- Optimization of Processing Speed
- Integration with Modern Technologies

6.2.6 Development Methods

For the development of our project, Scrum methodology will be used. Scrum is an agile development approach used to improve software that is dependent on repeatable and iterative cycles. Work will be divided into several sprints, each of which will have a deadline. This period will not exceed one week. A 15-to-20-minute meeting will be held following the completion of each sprint to review progress and adjust the overall schedule as necessary. Scrum is adaptable, making it simpler to change projects or add new functionalities, so it makes sense to use this development methodology.

6.3 System Architecture

This section provides an overview of our system architecture. Our system follows a layered architecture giving various advantages like increased abstraction, low dependency and coupling, easier modification and extension of the system.

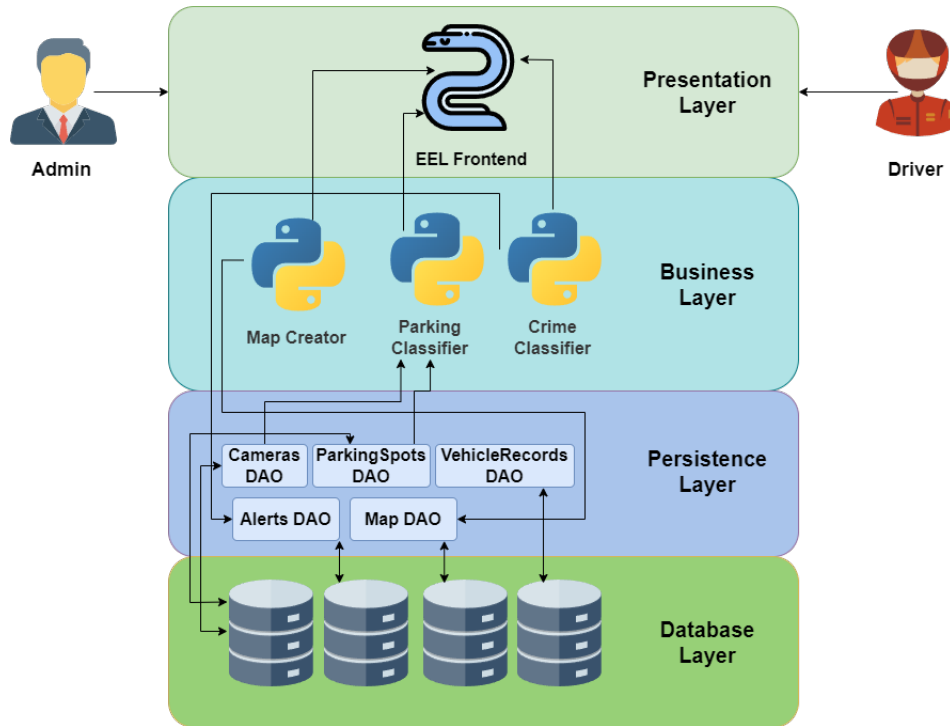


Figure 6.1: Architecture Diagram

Shows a high level overview of the system's architecture.

As shown in the diagram, our system utilizes layered architecture approach to separate different functionalities of the systems. The system is divided into 4 distinct layers namely the **Presentation Layer** responsible for handling the frontend of our project, providing an easy and intuitive way for users to interact with the system, the **Business Layer** which contains all the entities and their operations/interactions among one another, the **Persistence Layer** which is responsible for handling database operations like creation, update, delete, and read, and finally the **Database Layer** which will be used to store the data.

6.3.1 Subsystem Architecture

Architecture of some important modules of our system is presented in this section. All modules will be built in Python and TensorFlow will be used for implementing, training and testing modules using deep learning techniques as in Parking Lot Classification and Crime Detection.

6.3.1.1 Parking Spot Classification Module

This module is responsible for classification of parking spots in the lot. The architecture of our CNN is given in the figure below:

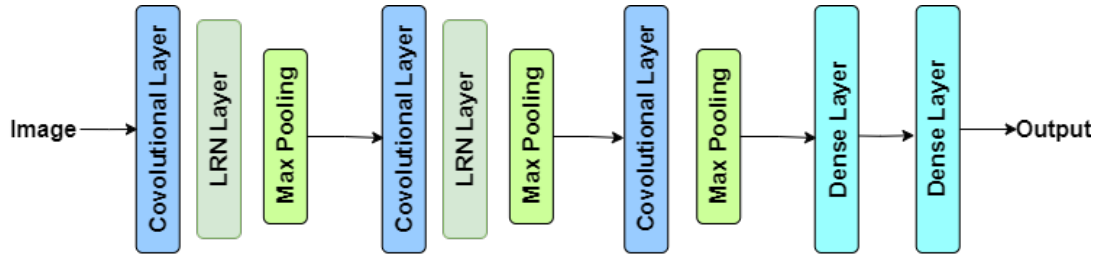


Figure 6.2: Architecture of Parking Classifier

Shows the architecture of the CNN used for parking spot classification.

The model uses 3 convolutional layers that are responsible for applying the convolution operation on the image with various kernels, each convolution layer uses ReLU activation function and it is followed by a max pooling layer that performs a pooling operations that selects the maximum element from the region of the feature map covered by the filter. First two convolutional layers are also followed by an LRN layer which applies Linear Response Normalization to normalize the results of the convolution. Finally, two Dense layers are used in the end to get the final output.

6.3.1.2 License Plate Recognition Module

This module is tasked with the detection of license plates and the extraction of license numbers from images. To accomplish this, we have utilized the YOLOv8 object detection model for license plate localization, then a filter is convolved over the cropped license plate image to enhance the letters, and then EasyOCR, a widely recognized pre-trained character recognition library available for python, is used for text extraction. This model's advanced capabilities ensure accurate and efficient license plate recognition, contributing to the overall functionality of our system.

6.3.1.3 Optimal Routing Module

This module is designed to calculate the shortest route from the entrance to an available parking spot, and it communicates this route to the front-end module for straightforward visualization. The A* Algorithm, a heuristic-based path calculation algorithm renowned for its efficiency and accuracy, is employed for the computation of the shortest route. This ensures that users are provided with the most convenient path to their designated parking spot, enhancing the overall user experience.

6.3.1.4 Crime Detection Module

This module also incorporates a Convolutional Neural Network (CNN) in conjunction with a Long Short-Term Memory (LSTM) network for information comparison with previous frames. The CNN is used for feature extraction from the images, while the LSTM helps in maintaining the temporal context between frames. This combination allows for more accurate and efficient processing, particularly in situations where the context from previous frames is crucial for understanding the current frame. The architecture is given in the following figure:

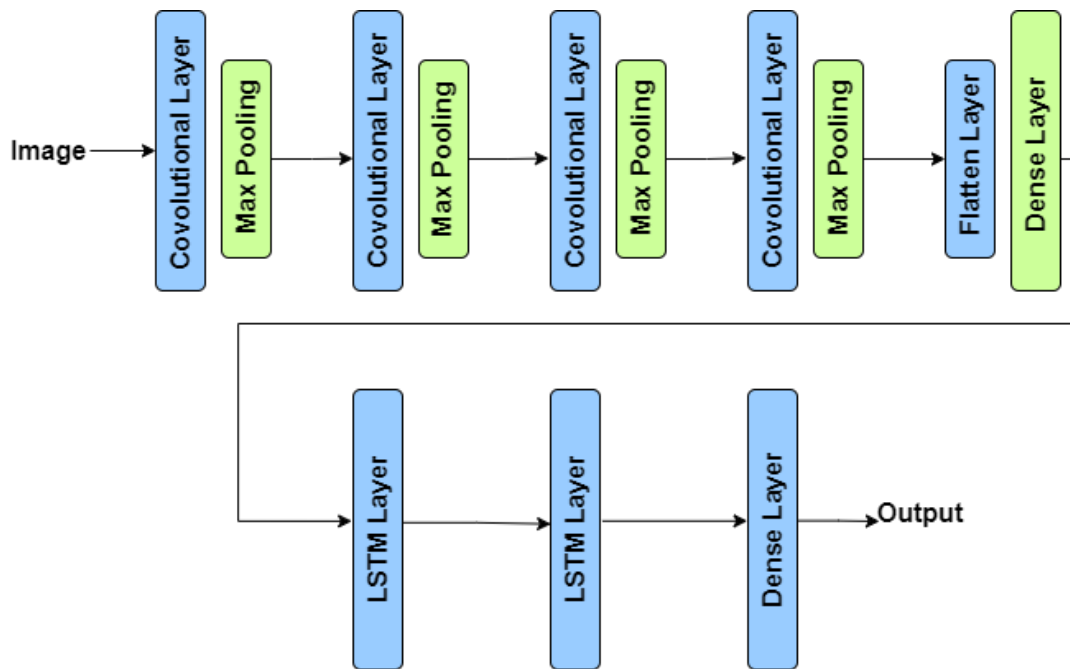


Figure 6.3: Architecture of Crime Classifier

Shows the architecture of the CNN+LSTM used for crime classification.

6.4 Architectural Strategies

The architectural strategies we aim to use are following:

6.4.1 Programming Language

For the frontend design we are using the EEL framework. EEL framework makes frontend development very efficient by integrating Python with HTML, CSS, and JavaScript, allowing a efficient development process. Its integration of web technologies facilitates the creation of responsive user interfaces, making it user-friendly choice for desktop application development. For machine learning process we are using the TensorFlow which is an open source machine learning framework for everyone. TensorFlow is mostly used for building and training machine learning models, particularly in the field of deep learning.

Additionally, for the camera calibration and image pre-processing we are using the OpenCV which is an open-source computer vision and machine learning software library.

6.4.2 Future plans for extending or enhancing the software

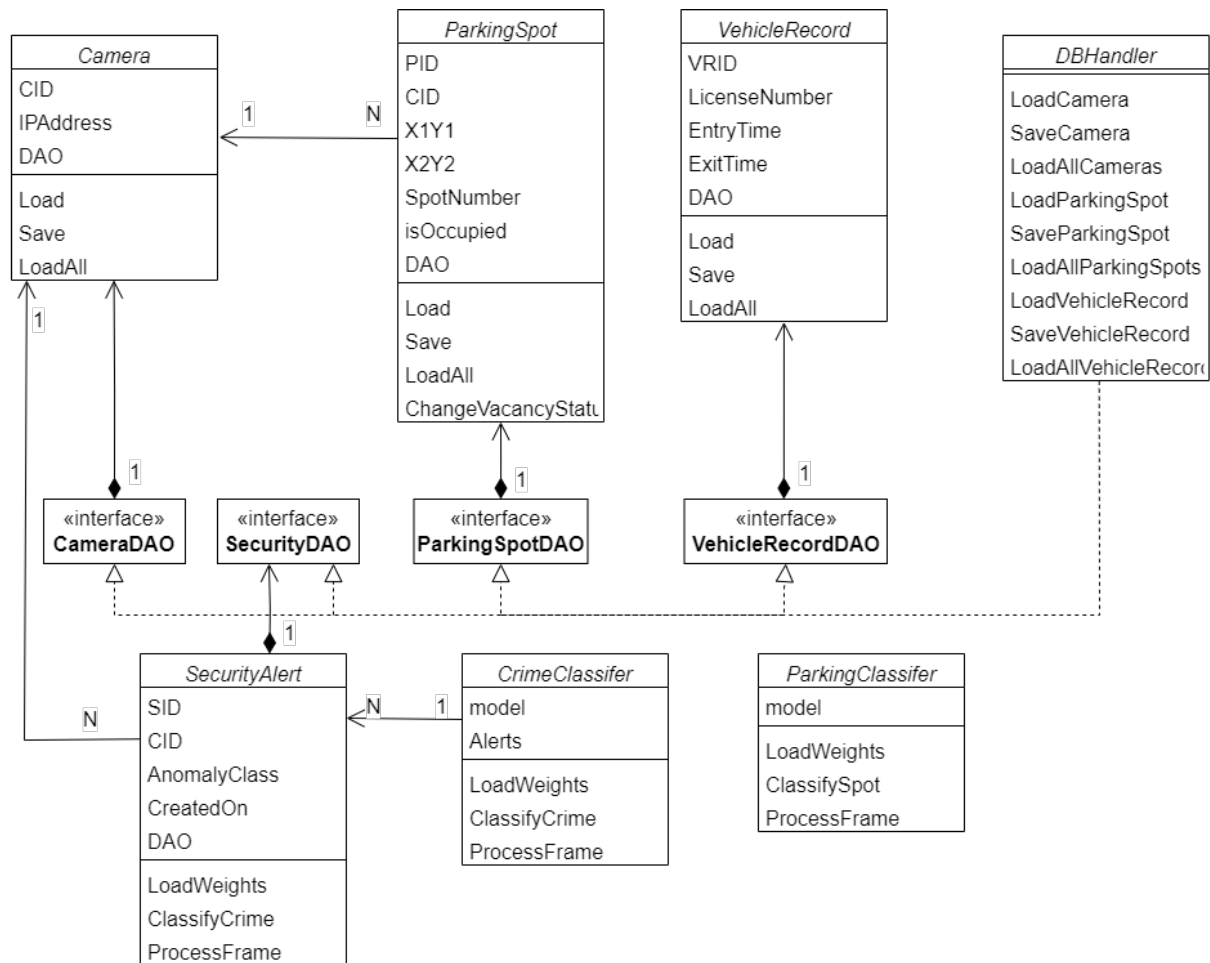
As far as our plan for extending or enhancing the software is concerned, we aim to make our system so efficient that it can perform real-time segmentation. Additionally, we plan to develop a web application to accommodate users, allowing them to view a dynamically updated real-time map on their mobile phones.

6.4.3 Concurrency and Synchronization

Assigning each camera to a different thread in our car parking system helps process images simultaneously, reducing delays and improving responsiveness. This approach also allows efficient utilization of computer resources, making the system better equipped to handle multiple tasks concurrently.

6.5 Domain Model/Class Diagram

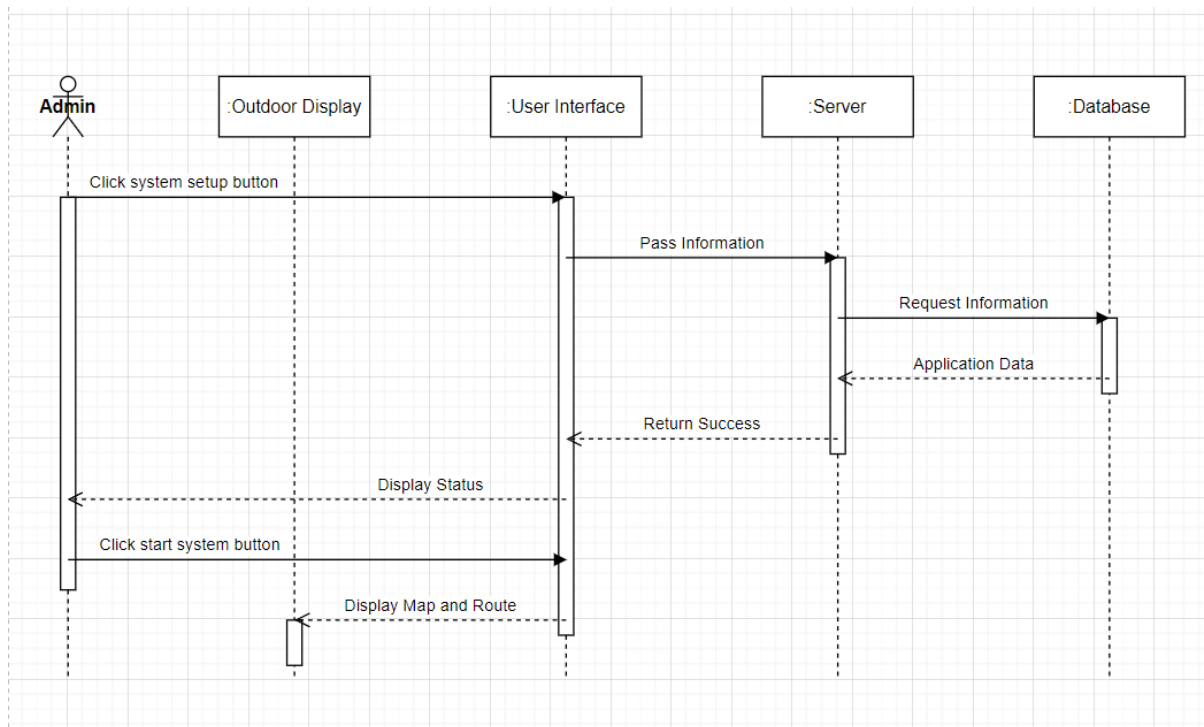
The subsequent figure provides a representation of the class diagram for our system. This diagram serves as a blueprint, detailing the structure and interactions of the classes within our system. It is an essential tool for understanding the overall architecture and relationships within the system.

**Figure 6.4: Class Diagram**

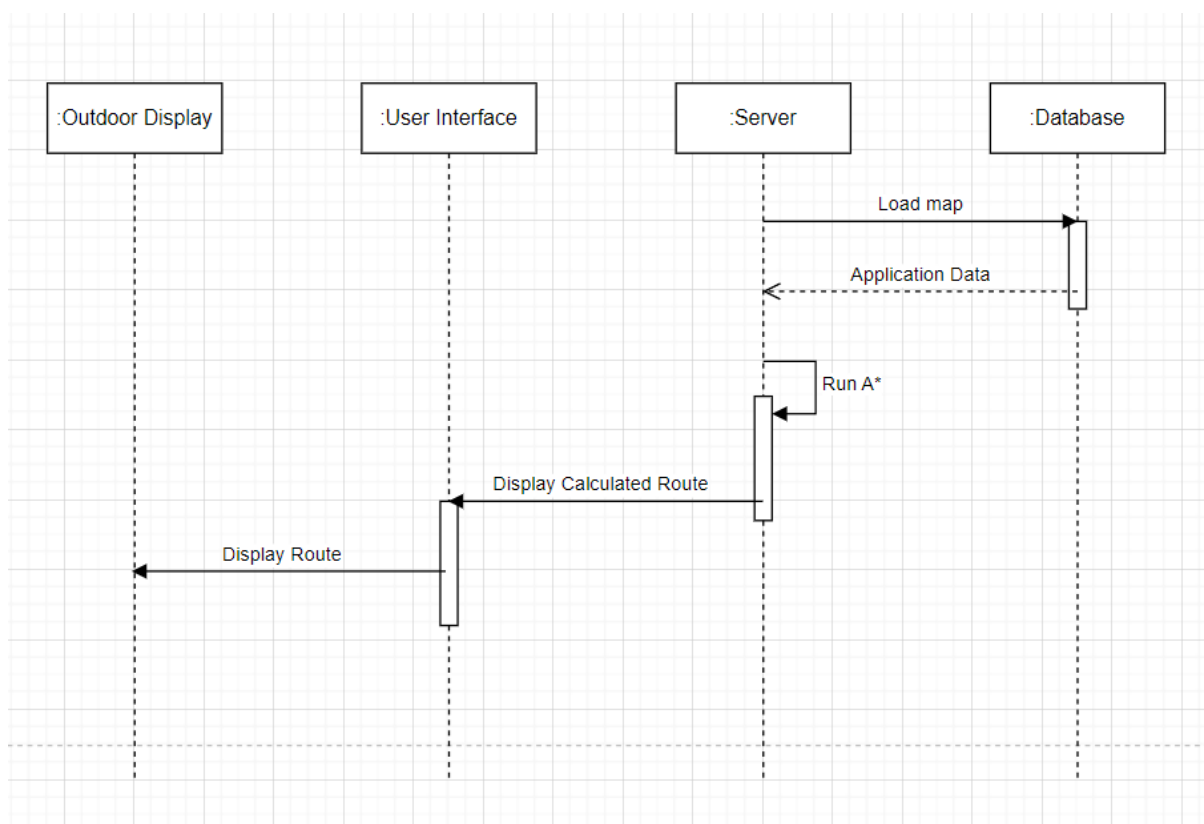
This class diagram describes the models and their relations.

6.6 Sequence Diagrams

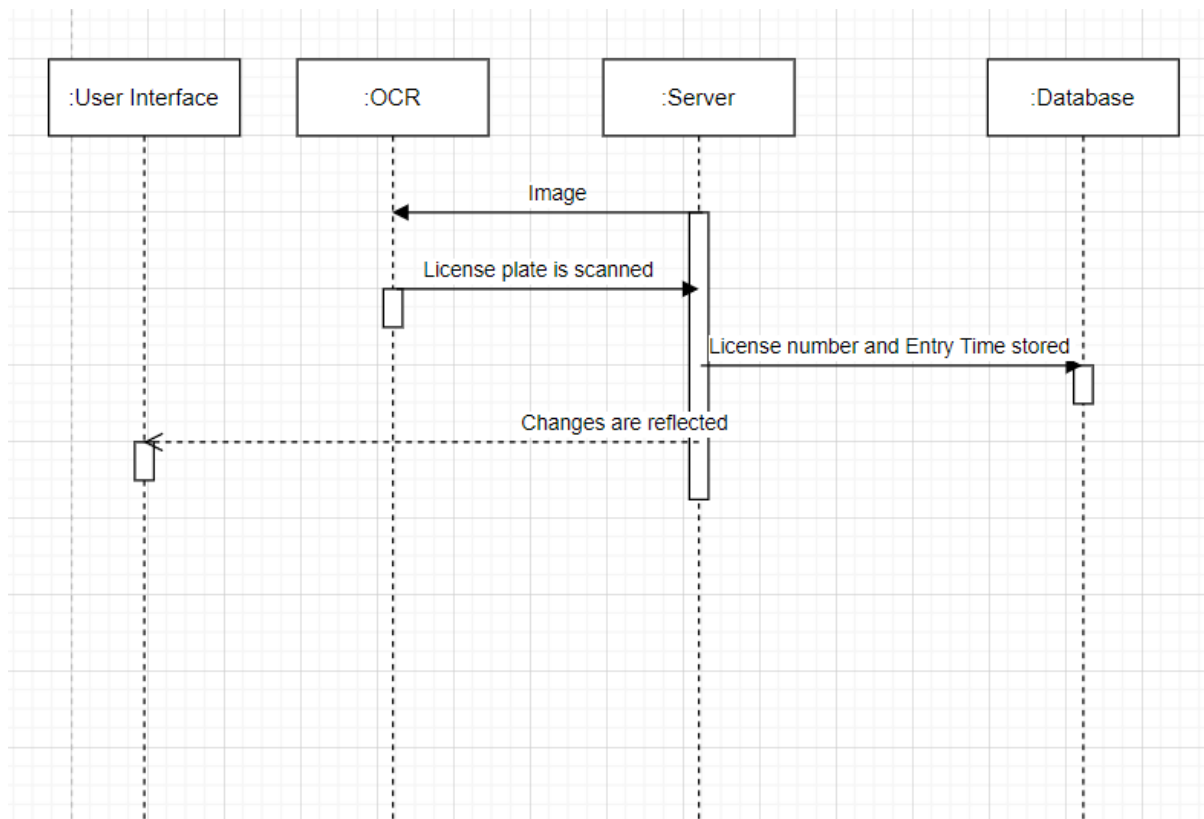
A sequence diagram is a type of UML (Unified Modeling Language) diagram that depicts interactions and messages exchanged segments of a system. It provides a dynamic view of a system. It highlights the timing of event by using activation bar. Sequence diagrams are particularly useful for visualizing and understanding the continuity of activities.

**Figure 6.5: Sequence of System Setup**

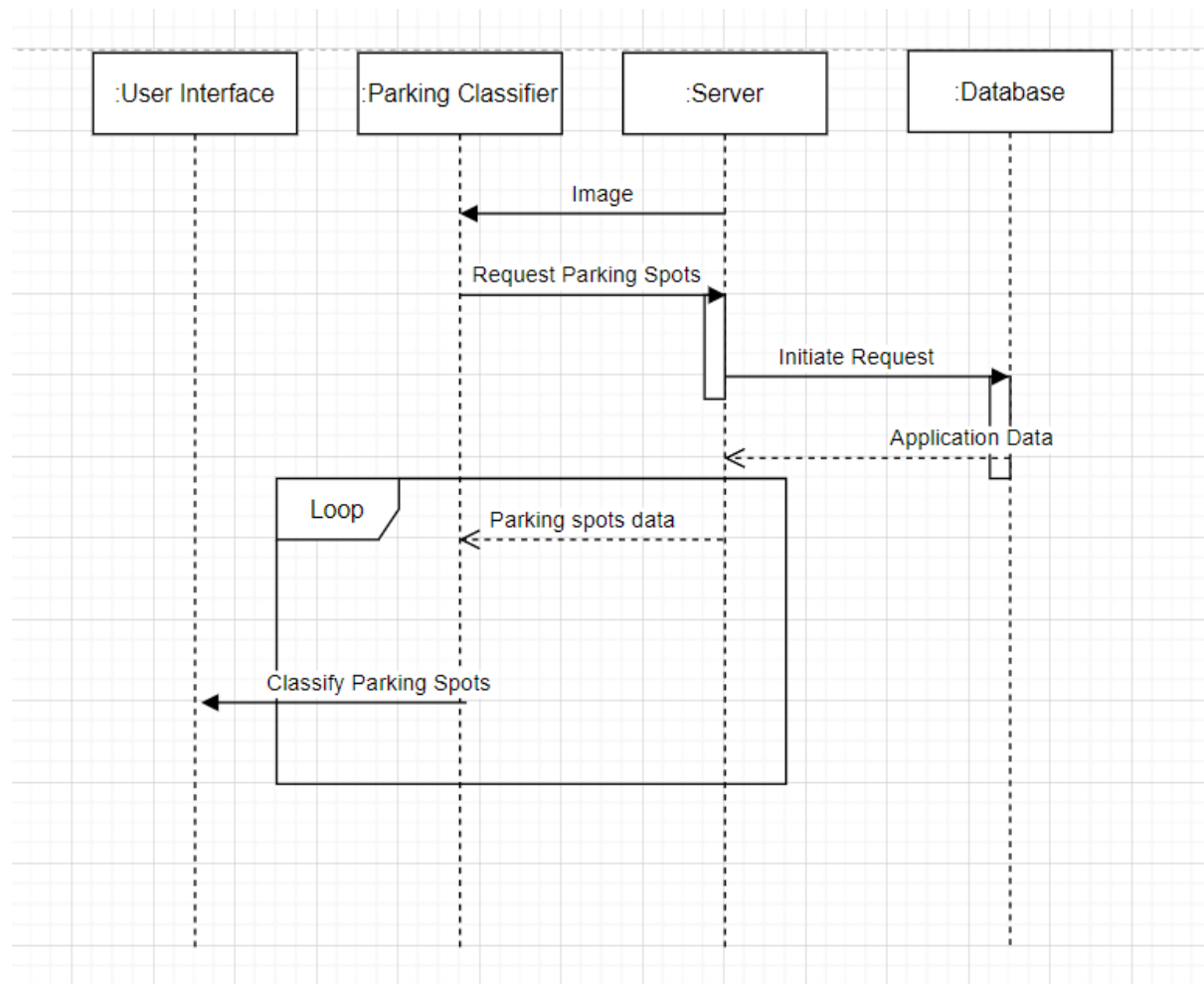
Flow/Sequence of operations followed by the system during setup.

**Figure 6.6: Sequence of Route Update**

Flow/Sequence of operations followed by the system to calculate and display shortest route for drivers' convenience.

**Figure 6.7: Sequence of Vehicle Entry/Exit**

Sequence of instructions followed by the system whenever a vehicle enters or exits the lot.

**Figure 6.8: Sequence of Parking Classification**

Sequence of operations performed by the system to classify all the spots in the image.

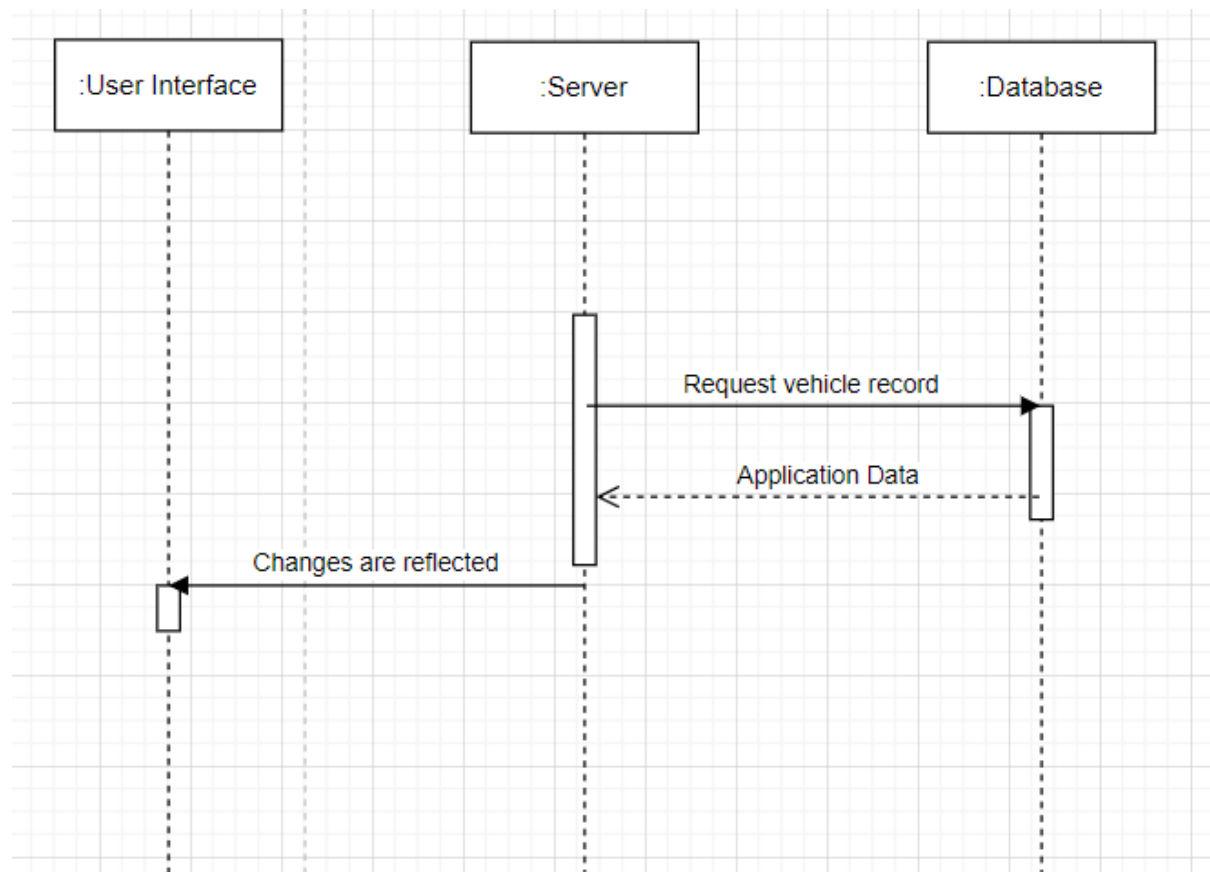


Figure 6.9: Sequence of Displaying Vehicle Record

Admin can see all the vehicle record on the system main screen. This will help to retrieve any vehicle information that entered the parking lot.

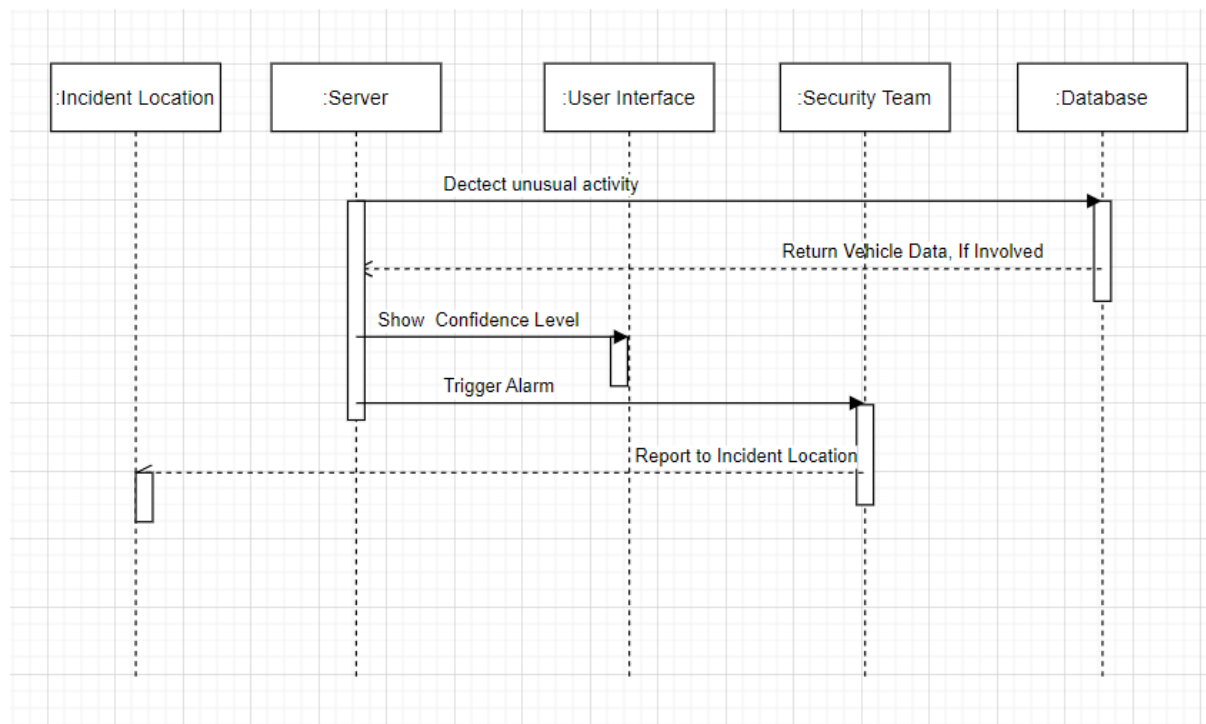


Figure 6.10: Sequence of Crime Detection

Crime detection is done by our system. Our system will alert the security team. The confidence level will be shown on the main system screen.

6.7 Policies and Tactics

The following are the considerations for the policies and tactics that would be followed in the project.

6.7.1 Coding guidelines and conventions

As we are using python libraries such as Open CV and TensorFlow for our project. Thus, there are some standard rules such as naming convention, formatting standards, and code organization. These rule will be followed in our project.

6.7.2 User Interface

The application will be simple yet modern and easy to use. It will ensure that an appealing color scheme is used that is relevant and right. There will be detailed interface design for the end user.

6.7.3 Extensibility

System's extensibility will be kept in mind. We will use modular approach, so if we want to add new features in future, it will be easy for us. Therefore, we will design our system accordingly. Thus, it will improve the integration ability of our system.

6.7.4 System Testing

Before the deployment process, both black box and white box testing will be done. This is to ensure that our system is free from any vulnerabilities. Thus, proper testing will be done and then the system will be deployed.

6.7.5 Engineering Trade-offs

While making design decisions, there will be trade-off between many factors. Such factors include system performance, resource utilization, and development time. For making informed decisions that align with our project goals. We need to identify and document these trade-offs as it is essentials for our decision making.

Chapter 7 Implementation and Test Cases

In this chapter, we will identify how we implemented our proposed idea. We will discuss how we converted our raw data into meaningful data. After that, we will see how these images were processed and used to train our models.

7.1 Parking Spot Classification Model

Parking Spot classification is used to identify and classify each empty and occupied parking spot in the parking lot. In this section, we will discuss custom CNN approach and cmAlexNet approach for parking spot classification.

7.1.1 Custom CNN Approach

This model uses 9 convolutional layers, with each layer followed by a max pooling layer. First convolutional layer used 16 kernel of size 5x5 and remaining layers used 3x3 kernels with 32 kernels used in seconds layer, 16 in third layer and 8 in remaining layers. Padding was used in all layers and stride was set to 1. Finally, two fully connected layers were used with 64 units in first and a single output neuron in the last layer. RELU and Sigmoid activations were used in the last two layers respectively. PKLot[6] dataset was used for training. Due to large size of the dataset, unavailability of GPU and risk of overfitting, the model was only trained for 2 epochs. ADAM optimizer and binary crossentropy loss function was used. 70, 10, 20 split was used for training, validation and testing sets. After training, the Tensorflow model obtained was converted to TensorFlow Lite which creates a light weight model speeding up the execution time.

7.1.2 cmAlexNet Approach

This model uses 3 convolutional layers, with each layer followed by a max pooling layer. First 2 convolutional layers also have a Local Response Normalization Layer before the max pooling layer. First convolutional layer used 16 kernel of size 13x13, and a stride of 4. Second convolutional layer used 20 kernels of size 5x5, and a stride of 1. Third convolutional layer used 30 kernels of size 3x3, and a stride of 1. Padding was used in all layers and relu activation was used. All max pooling layers also use a stride of 2. Finally, two fully connected layers were used with 48 units in first and a single output neuron in the last layer. RELU and Sigmoid activations were used in the last two layers respectively. All layers also use L2 regularization with lambda set to 0.00025. PKLot[6] dataset was used for training. This model was only trained for 3 epochs. ADAM optimizer and binary crossentropy loss function was used. 70, 10, 20 split was used for training, validation and testing sets. After training, the Tensorflow

model obtained was converted to TensorFlow Lite which creates a light weight model speeding up the execution time.

7.2 Crime Detection Model

Criminal activities in the parking lot will be detected and security personnel will be notified about that unusual activity. This model uses 3 convolutional layers, with each layer followed by a max pooling layer. First convolutional layer used 64 kernel of size 3x3, second convolutional layer used 128 kernels of size 3x3, third convolutional layer used 256 kernels of size 3x3. Padding was used in all layers and LeakyReLU activation was used. A single fully connected layers were used with 256 units which is then connected to 2 LSTM layers with 8 units each. This is proceeded by a Dense layer with 4 units using Tan Hyperbolic activation and finally a Dense layer with 14 units is used as output layer. This uses the Softmax activation. UCF Crime[13] dataset was used for training. This model was only trained for 2 epochs. ADAM optimizer and categorical crossentropy loss function was used. 70, 10, 20 split was used for training, validation and testing sets.

7.3 Automatic Number Plate Recognition

To implement the ANPR module, we first created a dataset for training the YOLOv8 object detection model. For this, images were taken from the internet, and labelled using a labelling tool. Then, data augmentation techniques were used to increase the size and improve generalizability of the model. The augmentation techniques used involved image rotation, translation, crop and mirroring. After creating the dataset, the YOLOv8 model was trained for 10 epochs ensuring that the model was properly learning without showing high bias or variance. To extract the text from license plates, the image is first passed through the trained model which localizes the license plate and the image is cropped using the coordinates returned by the model. A margin of 5 pixels is used when cropping the image to account for any mistakes made by the model. Then, filters are applied to enhance the image. This includes a 3x3 laplacian filter to improve sharpness and a 3x3 filter to increase image brightness which removes unwanted smaller texts from the license plate. The image is then resized to 300x200 pixels and passed as an input to the EasyOCR model. The results returned by the EasyOCR model are then filtered to remove common unwanted words like state names, and rearranged in the form of a license plate number.

7.4 Spot Information

This module keeps a record of all the available and occupied spots in the parking lot. It shows the total available spots out of total number of spot. Apart from this there is a view of all the available and empty spots in rectangle shape. Occupied spots are marked as red and inside it there is spot number shown. Empty spot is marked green and inside it there is a spot number respectively. This screen is for admin side to check the occupation of parking lot.

7.5 Vehicle Information

This module keeps record of all the vehicles information. On Entrance, license plate of vehicles are scanned and then it extract the text using the ANPR module. This text is stored in our database. Apart from this entry and exit time of vehicle is stored in the database. On front end, there is a section named vehicle information. On clicking this tab, it will show the data table containing vehicle information including license plate number, entry time and exit time. This data table further include features such as searching, pagination and restricting number of records being shown. This will collectively help the authority for robust searching of vehicle data.

7.6 Set Credentials

This section include credentials that is username and password for the cameras. This is stored in the database including IP address. Then, these credentials are used for camera connectivity. Further, there is unique camera number and is-active which will show either camera is working or not.

7.7 Select Cameras

This section show all the cameras including entry and parking cameras. Entry camera is used for the license plate recognition and parking cameras is used for the parking classification. There is a feature available to toggle the camera on and off. If it is on then the feed of this camera will be utilized in parking classification. Otherwise, the respective camera will not be used.

7.8 Mark Parking Spots

This section include the selection or marking of parking spot on admin side. There are images shown of the parking lot. To select parking spot from the image, we have to draw rectangle by selecting 2 diagonal points according to the parking spot in the image. This is then passed to the cmAlexNet model,

which will identify occupied and empty spots. This data is also stored in database which include spot x, y position and is-active which tell either the spot is empty or occupied.

7.9 Route Calculation

The system employs the A* algorithm to efficiently direct drivers to the nearest available parking spot. This choice is because of A* algorithm speed and accuracy in path finding. This makes it ideal for real time applications. When a change in parking spot occupancy is detected, the system references the already segmented positions that was stored in the database. Initially, the system segments the parking spots and saves these positions in the database. Various enhancement filters such as binary conversion and sharpness filters are applied to the uploaded map image. This image is then saved on the server. Using this enhanced map image, the A* algorithm calculates the shortest path from the driver's current location to the closest available spot. This path is then visually represented on the main screen. This provides clear guidance for the driver. The combination of rapid detection and A*'s efficiency ensures a seamless parking experience.

7.10 Test Case Design and Description

This section explains the test cases designed to test the functionality of our system. Details of each test case are given below.

7.10.1 Parking Spot Classification

Table 7.1: Parking Spot Classification Test Case

Tests related to how well the system can classify given parking spots as either occupied or available.

Parking Spot Classifier			
Test Case ID:	1	QA Test Engineer:	Zain Ahmad
Test case Version:	1	Reviewed By:	Ali Raza
Test Date:	29-04-2024	Use Case Reference(s):	None
Revision History:	None		
Objective:	Test the accuracy of parking spot classifier		
Product/Ver/Module:	ParkingSpotClassifier Module		
Environment:	Application is running on the system and some cameras overlooking the parking space are attached.		
Assumptions:	None		
Pre-Requisite:	User has finished the setup process selecting the desired cameras and segmenting the relevant parking spaces		
Step No.	Execution description	Procedure result	
1	User clicks on Start System button on the home page.	System takes user to the camera feed screen and shows the footage of all attached cameras, also starts the parking, anomaly and ANPR classification modules in the backend.	
2	User clicks on Expand Navbar button on the home page.	System expands the Navbar showing user 4 buttons that are Camera Feed, Vehicle Information, Spots Information and Security Alert.	
3	User clicks on Spots Information button on the Navbar.	System takes user to the Spots Information screen where it shows the user all the available and occupied parking spots colored as green and red respectively.	
Comments: System correctly displayed the status of all parking spots.			
Passed			

7.10.2 Anomaly Classification

Table 7.2: Anomaly Classification Test Case

Tests related to how well the system can classify given a scene into various anomaly classes as specified in the module.

Anomaly Classifier			
Test Case ID:	2	QA Test Engineer:	Zain Ahmad
Test case Version:	1	Reviewed By:	Ali Raza
Test Date:	29-04-2024	Use Case Reference(s):	None
Revision History:	None		
Objective:	Test the accuracy of anomaly classifier		
Product/Ver/Module:	AnomalyClassifier Module		
Environment:	Application is running on the system and some cameras overlooking the parking space are attached.		
Assumptions:	None		
Pre-Requisite:	User has finished the setup process selecting the desired cameras and segmenting the relevant parking spaces		
Step No.	Execution description	Procedure result	
1	User clicks on Start System button on the home page.	System takes user to the camera feed screen and shows the footage of all attached cameras, also starts the parking, anomaly and ANPR classification modules in the backend.	
2	User clicks on Expand Navbar button on the home page.	System expands the Navbar showing user 4 buttons that are Camera Feed, Vehicle Information, Spots Information and Security Alert. The system highlights the Security Alert button in red whenever it detects any anomalies.	
3	User clicks on Security Alert button on the Navbar.	System takes user to the Security Alerts screen where it shows the user all the current anomalies detected. The screen shows anomaly type and module confidence.	
Passed			

7.10.3 Automatic Number Plate Recognition

Table 7.3: License Plate Localization Test Case

Tests related to how well the system can identify and localize license plates in an image.

License Plate Localization			
Test Case ID:	3	QA Test Engineer:	Zain Ahmad
Test case Version:	1	Reviewed By:	Ali Raza
Test Date:	29-04-2024	Use Case Reference(s):	None
Revision History:	None		
Objective:	Test the accuracy of license plate localization in an image		
Product/Ver/Module:	ANPRClassifier Module		
Environment:	Application is running on the system and some cameras overlooking the parking space are attached.		
Assumptions:	None		
Pre-Requisite:	User has finished the setup process selecting the desired cameras and segmenting the relevant parking spaces		
Step No.	Execution description	Procedure result	
1	User clicks on Start System button on the home page.	System takes user to the camera feed screen and shows the footage of all attached cameras, also starts the parking, anomaly and ANPR classification modules in the backend. Here, the system sends images to ANPR module which uses the trained YOLOv8 model to localize license plates in the image	
Passed			

Table 7.4: License Plate Recognition Test Case

Tests related to how well the system can identify and convert license plates in an image to text.

License Plate Text Extraction			
Test Case ID:	4	QA Test Engineer:	Zain Ahmad
Test case Version:	1	Reviewed By:	Syed Hassan Haider
Test Date:	29-04-2024	Use Case Reference(s):	None
Revision History:	None		
Objective:	Test the accuracy of license plate to text conversions		
Product/Ver/Module:	ANPRClassifier Module		
Environment:	Application is running on the system and some cameras overlooking the parking space are attached.		
Assumptions:	None		
Pre-Requisite:	User has finished the setup process selecting the desired cameras and segmenting the relevant parking spaces		
Step No.	Execution description	Procedure result	
1	User clicks on Start System button on the home page.	System takes user to the camera feed screen and shows the footage of all attached cameras, also starts the parking, anomaly and ANPR classification modules in the backend. The system sends images to ANPR module which uses the trained YOLOv8 model to localize license plates in the image, it is then pre-processed with several filters and then used as an input to Easy-OCR which extracts text. Text is then filtered and ordered to get the final license plate number.	
Passed			

7.10.4 Setup Screen

Table 7.5: Camera Selection

Tests related to how well the system can classify given parking spots as either occupied or available.

Camera Selection			
Test Case ID:	5	QA Test Engineer:	Ali Raza
Test case Version:	1	Reviewed By:	Syed Hassan Haider
Test Date:	29-04-2024	Use Case Reference(s):	None
Revision History:	None		
Objective:	Test the accuracy of camera selection		
Product/Ver/Module:	Camera Repository		
Environment:	Application is running on the system and all the camera are shown to admin for selection.		
Assumptions:	There are camera's available for feed.		
Pre-Requisite:	User have given the credentials of IP Address of cameras		
Step No.	Execution description	Procedure result	
1	User clicks on Setup System button on the home page.	System takes user to the camera feed screen and shows the footage of all attached cameras.	
2	User marks the check-box of different camera for parking.	System saves all the selected cameras for parking.	
3	User hits the Save and Next button.	System redirects the user to next page for indoor camera selection and user select a camera for entry level .	
4	User clicks the Save button.	System saves all the cameras selected for parking and indoor in the camera repository.	
Passed			

Table 7.6: Spots Segmentation

Tests related to how well the system can classify given parking spots as either occupied or available.

Spots Segmentation			
Test Case ID:	6	QA Test Engineer:	Ali Raza
Test case Version:	1	Reviewed By:	Syed Hassan Haider
Test Date:	29-04-2024	Use Case Reference(s):	None
Revision History:	None		
Objective:	Test the accuracy of spots marking		
Product/Ver/Module:	Spots Repository		
Environment:	Application is running on the system and all the camera are shown to admin for spots marking.		
Assumptions:	User has selected the parking cameras.		
Pre-Requisite:	User has selected the parking cameras.		
Step No.	Execution description	Procedure result	
1	User clicks on Setup System button on the home page.	System takes user to setup screen.	
2	User click the Nav bar button.	System shows all the options of Nav bar.	
3	User clicks the Spots Marking button.	System redirects the user to the spots marking page.	
4	User marks the spots.	System takes all the spots marked by the user and save them and repeat the process for all the selected parking cameras..	
5	User hits the Save button.	System save all the spots marked by the user along with the camera id of each camera and save it to the spots repository.	
Passed			

7.10.5 Start Screen

Table 7.7: Camera Feed

Tests related to how well the system can classify given parking spots as either occupied or available.

Camera Feed			
Test Case ID:	7	QA Test Engineer:	Syed Hassan Haider
Test case Version:	1	Reviewed By:	Ali Raza
Test Date:	29-04-2024	Use Case Reference(s):	None
Revision History:	None		
Objective:	Test the accuracy of camera feed.		
Product/Ver/Module:	Camera Repository		
Environment:	Application is running on the system and all the selected cameras are shown to user.		
Assumptions:	User has selected the cameras.		
Pre-Requisite:	User has selected the cameras.		
Step No.	Execution description	Procedure result	
1	User clicks on Start System button on the home page.	System takes the user to the start system page.	
2	User click the Nav Bar button.	System shows the user all the option of start system.	
3	User clicks the Camera Feed Button	System shows feed of all the cameras marked by the user in setup screen for parking.	
Passed			

Table 7.8: Vehicle Information

Tests related to how well the system can classify given parking spots as either occupied or available.

Vehicle Information			
Test Case ID:	8	QA Test Engineer:	Syed Hassan Haider
Test case Version:	1	Reviewed By:	Ali Raza
Test Date:	29-04-2024	Use Case Reference(s):	None
Revision History:	None		
Objective:	Test the accuracy of vehicle entry information.		
Product/Ver/Module:	Vehicle Repository		
Environment:	Application is running on the system and there are records of vehicles.		
Assumptions:	None		
Pre-Requisite:	None		
Step No.	Execution description	Procedure result	
1	User clicks on Start System button on the home page.	System takes the user to the start system page.	
2	User click the Nav Bar button.	System shows the user all the option of start system.	
3	User clicks the Vehicle Information Button	System fetches the record of all the vehicles with their entry time. User can also search for a specific record.	
Passed			

7.11 Test Metrics

This section highlights some metrics for our test cases. These are explained in Table 7.9 below.

Table 7.9: Test case Metrics

Various test case metrics like no. of total cases executed, failed test cases, and test case effectiveness.

Metric	Purpose
Number of Test Cases	8
Number of Test Cases Passed	8
Number of Test Cases Failed	0
Test Case Defect Density	0
Test Case Effectiveness	0

Chapter 8 User Manual

The following are the steps which will be followed by the user to setup the system and then start it.

8.1 System Setup

After running the desktop application, the first step is to click the system setup button. After entering the system setup, there will be a navigation bar on the left side of the screen in which few options will be available which user has to follow sequentially. The options are discussed below.

8.1.1 Camera Selection

After setting the credentials of camera. User has to select the cameras. There are 2 variants of camera. First one is entry camera which user will select by clicking the checkbox. After this, user will click save and next button given on the bottom right corner of the screen. this will leads to the selection of parking cameras. In this, same procedure will be followed in camera selection by clicking the checkbox. This completes our first phase of system setup.

8.1.2 Parking Spot Segmentation

After the successful completion of first phase, user will now segment the parking spots using the camera feed. In this process, user will mark rectangles by clicking the spot 2 times most probably in diagonal way such that a rectangle is formed around the required single car parking space. This process will be repeated until all the parking spots are marked. If a wrong rectangle is drawn in segmentation, user can simply right click inside the rectangle to remove it and redraw a new one by clicking 2 times in a diagonal format to form a rectangle. After this process is completed, our system is ready to be started.

8.2 Start System

After system setup, user will simply click the start system button which will start our system and user will have the opportunity to explore all the features in our system.

8.2.1 Camera Feed

In this screen, user will be able to see all the selected cameras. There will be a scroll-bar to scroll if there are multiple camera feeds. This can also be used to see live view in the parking lot by multiple cameras in a single screen.

8.2.2 Spots Information

This feature basically shows all the available and occupied spots in the parking lot. These spots will be shown as a rectangle with a spot number in the center. The available spots will be marked as green rectangles and the occupied spots will be marked as red rectangles. The total count of available and occupied spots will also be shown on the top of the screen.

8.2.3 Vehicle Information

This feature will allow the user to view all the vehicle information which includes license plate number, vehicle entry and exit time. This will be shown in a table which has multiple features such as search functionality, indexing, total records shown and entries per page can also be manipulated. This makes a whole package to explore and use for multiple scenarios.

8.2.4 Security Alert

This feature will enhance the security of parking lot as any unusual activity such as crime, fight, accidents will be shown to the user on the screen. This will also display the confidence level and type of unusual activity detection in the parking lot on the top of the screen.

Chapter 9 Experimental Results and Discussion

This chapter highlights the experiments conducted till now and their results. This chapter also highlights the hardware used and the model details.

9.1 Evaluation Metrics

This section details the evaluation metrics used to judge the models performance. Some abbreviations used to explain evaluation metrics are given below.

- **TP:** Stands for True Positives and it indicates the number of predictions made correctly that belong to the positive class.
- **TN:** Stands for True Negatives and it indicates the number of predictions made correctly that belong to the negative class.
- **FP:** Stands for False Positives and it indicates the number of negative class observations incorrectly predicted as positive.
- **FN:** Stands for False Negatives and it indicates the number of positive class observations incorrectly predicted as negative.
- **y:** Actual label of a given observation.
- **$\hat{y}$:** Label predicted by the model for a given observation.

9.1.1 Accuracy

Accuracy defines the correctness of model's predictions. It evaluates the ratio of correct predictions to total predictions made by the model.

Mathematically, it can be defined as:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (9.1)$$

9.1.2 Precision

Precision is another metric used to evaluate a models performance. It indicates the quality of positive predictions made by the model. Mathematically, it can be defined as the ratio of true positives by the total number of positive predictions made by the model.

$$Precision = \frac{TP}{TP + FP} \quad (9.2)$$

9.1.3 Recall

Next evaluation metric used is recall which describes the actual number of true positives identified by the model. Meaning, recall is defined as the number of true positives divided by the total actual positives. It measures the ability of the model to correctly identify the positive class.

$$Recall = \frac{TP}{TP + FN} \quad (9.3)$$

9.1.4 F1-Score

F1 score judges the overall performance of the model based on its recall and precision. It is defined as the harmonic mean of model's precision and recall and it can be calculated using this formula:

$$F1-Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (9.4)$$

9.1.5 Binary Cross Entropy

Binary cross Entropy measures the overall loss of a binary classification model. It calculates the dissimilarity between actual labels and predicted labels. Mathematically, it is given by:

$$Binary\ Cross\ Entropy = -(y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})) \quad (9.5)$$

9.2 Experimentation Setup

The experiments were performed on our local laptop running Jupyter Notebook. The hardware specification of the machine include an AMD Ryzen 7 5700U processor which has 8 physical cores and 16 logical cores using hyperthreading, 12 GB of ram, and no GPU. Due to the unavailability of GPU in our system, no hardware acceleration was used during training and testing.

9.3 Results

This section provides a detailed overview of the results that have been achieved in our study. It emphasizes the key findings and outcomes derived from our research, offering a comprehensive understanding of the implications and significance of these results.

9.3.1 Parking Spot Classification

This sections provides an overview of the models used for parking spot classification and results obtained after thorough testing of these models..

9.3.1.1 Custom CNN Model Approach

The following confusion matrix provides a comprehensive summary of the results derived from the model evaluation process. It encapsulates the performance metrics of the model, offering a clear depiction of its predictive capabilities and accuracy.

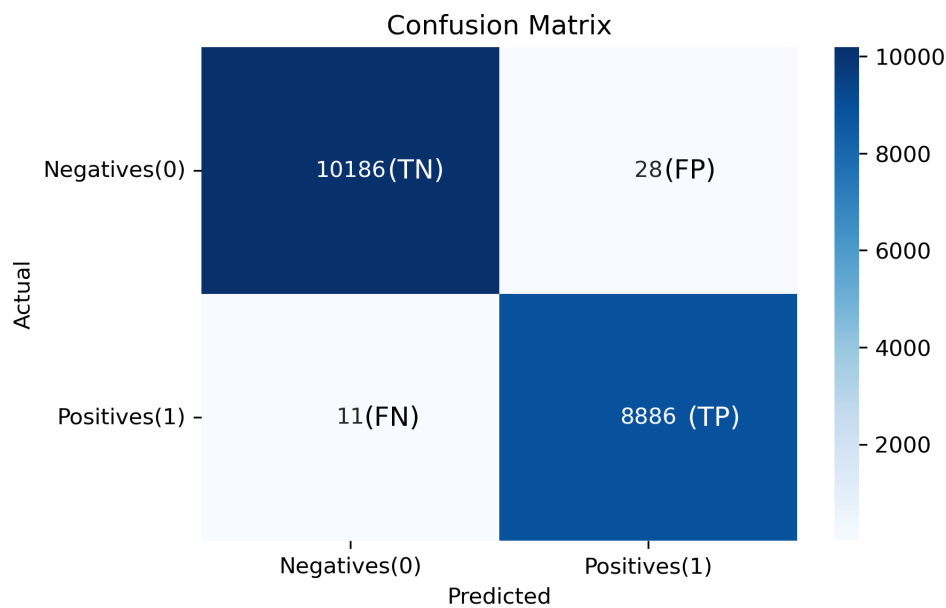


Figure 9.1: Confusion Matrix for Test Results Custom CNN

Confusion matrix created after evaluating the model on test dataset.

Following table highlights the evaluation metrics of the model discussed above and it also highlights the time taken for classification.

Table 9.1: Evaluation Metrics Performance Custom CNN

Performance of the model using evaluation metrics like Accuracy, Precision, Recall, F1, and time taken.

<i>Evaluation Metric</i>	<i>Score</i>
Accuracy	0.998031636
Precision	0.997009118
Recall	0.998764118
Binary Cross Entropy	0.008129032
F1 Score	0.997885841
Average Processing Time Per spot	0.007458157 seconds
Classifications Made Under 1 Second	140

9.3.1.2 cmAlexNet Model Approach

This section highlights the results and some insights obtained by testing the custom mAlexNet model on the test set after training. Models performance is visualized in the form of a confusion matrix given below:

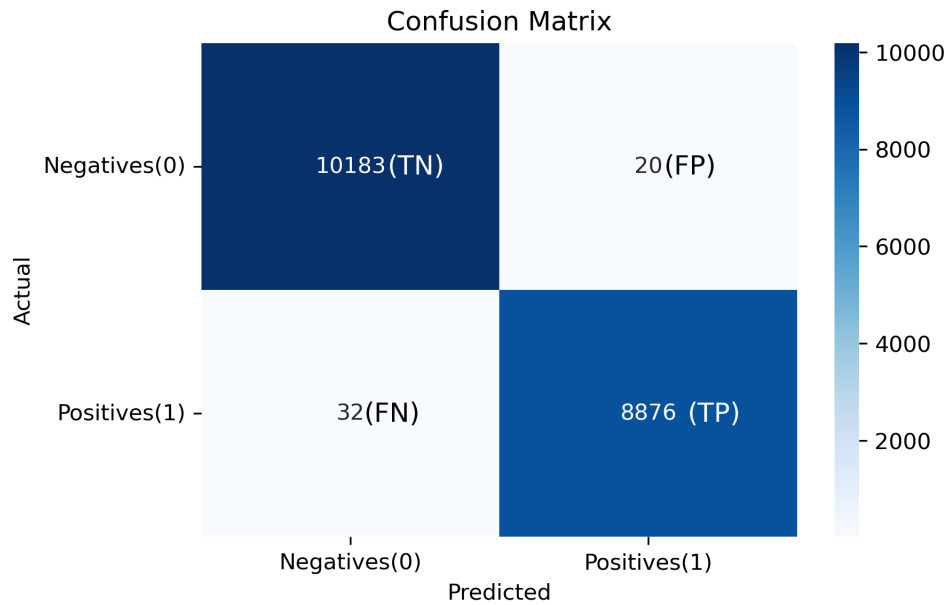


Figure 9.2: Confusion Matrix for Test Results cmAlexNet

Confusion matrix created after evaluating the model on test dataset.

Following table highlights the evaluation metrics of the model discussed above and it also highlights the time taken for classification.

Table 9.2: Evaluation Metrics Performance cmAlexNet

Performance of the model using evaluation metrics like Accuracy, Precision, Recall, F1, and time taken.

<i>Evaluation Metric</i>	<i>Score</i>
Accuracy	0.9972652
Precision	0.997714301
Recall	0.996407608
Binary Cross Entropy	0.009771032
F1 Score	0.997060526
Average Processing Time Per spot	0.003924215 seconds
Classifications Made Under 1 Second	260

Though this approach has negligibly worse loss due to slightly more false negatives predicted, it handily outperforms the first model when it comes to total time taken which is extremely crucial for a real-time system. It almost halves the processing time of given observation leading to almost twice the number of predictions made under 1 second.

9.3.2 Crime Detection

The following confusion matrix provides a summary of the results obtained from model evaluation. It encapsulates the key performance indicators of the model, offering a detailed look of its predictive accuracy and efficiency.

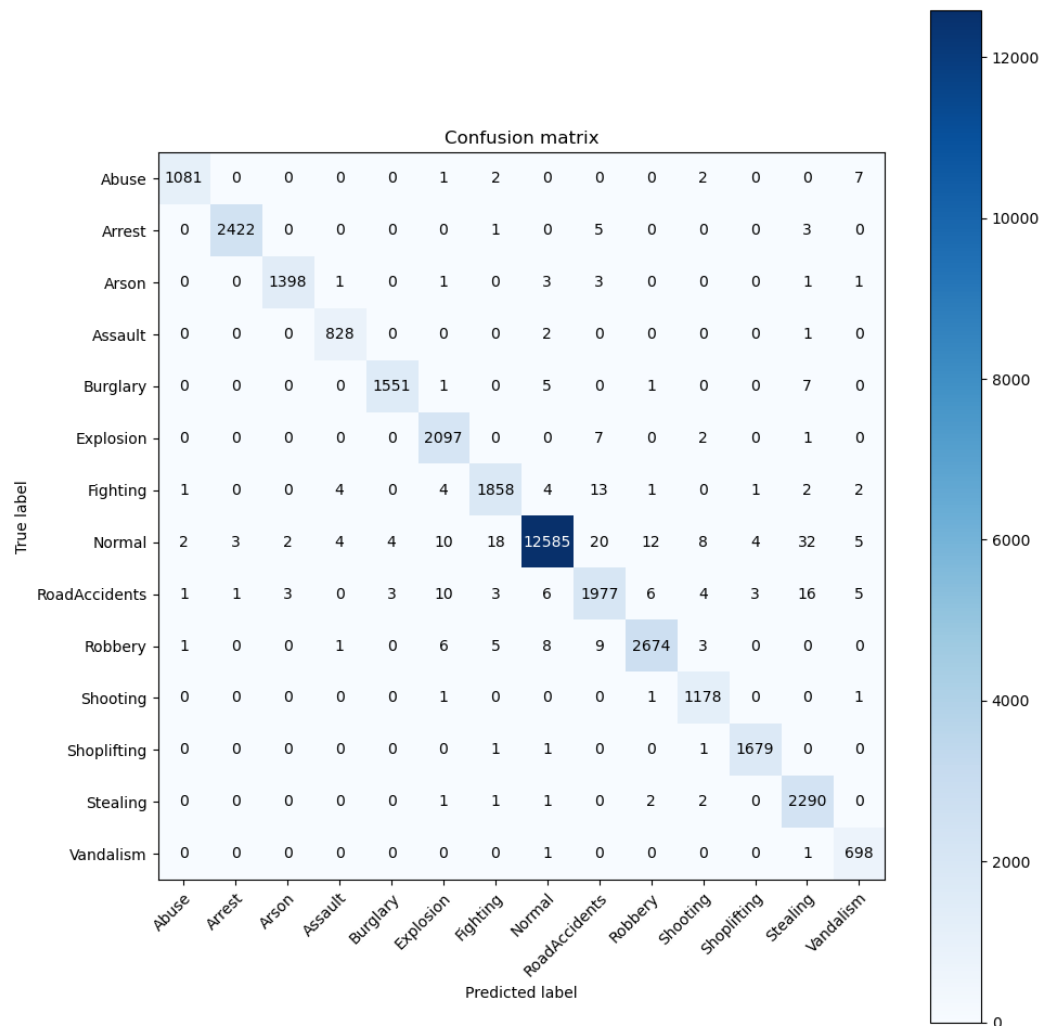


Figure 9.3: Confusion Matrix for Test Results Crime Detection

Confusion matrix created after evaluating the model on test dataset.

Following table highlights the evaluation metrics of the model discussed above and it also highlights the time taken for classification.

Table 9.3: Evaluation Metrics Performance Crime Detection

Performance of the model using evaluation metrics like Accuracy, Precision, Recall, F1, and time taken.

<i>Evaluation Metric</i>	<i>Score</i>
Accuracy	0.9907
Precision	0.990746
Recall	0.990675
Categorical Cross Entropy	0.03793253377079964
F1 Score	0.990686
Average Processing Time Per Frame	0.176672 seconds
Classifications Made Under 1 Second	6

Chapter 10 Conclusions

ParkEase is a comprehensive solution designed to provide hassle-free parking in parking lots. Leveraging computer vision techniques, we employ CCTV cameras to classify parking spots, eliminating the need for expensive sensors. At the entry level, users are presented with a display showing the available parking spots, facilitating a seamless parking experience.

The system not only classifies parking spots but also guides users to the nearest available spot, optimizing the parking process. This efficient approach not only enhances user convenience but also contributes to cost-effectiveness by utilizing existing infrastructure.

Beyond parking management, our system incorporates advanced security features. Using computer vision, we detect incidents such as car collisions or altercations, promptly notifying authorized personnel. This proactive approach ensures the timely awareness of unusual activities, contributing to a safer parking environment.

The project commenced with a meticulous exploration of the problem domain, establishing clear goals and objectives. Software requirements specifications were developed to detail both functional and non-functional aspects, promoting a systematic development approach. The proposed methodology outlined techniques for parking spot classification, optimal routing, and crime detection, providing a strategic roadmap for subsequent stages.

In the implementation phase, custom Convolutional Neural Network (CNN) and AlexNet models were employed for parking spot classification, with AlexNet exhibiting superior accuracy. The crime detection model demonstrated high-performance metrics, including accuracy, precision, recall, and F1 score. The precision, at 0.989980, reflects that nearly 99% of instances predicted as positive were indeed positive. A recall of 0.989925 indicates the model captured about 99% of all actual positive instances. The F1 score, harmonizing precision and recall, is reported as 0.989931, underscoring a balanced and high level of performance. ANPR module further fortifies both the utility and the security of our system by scanning and saving the license numbers of vehicle which is used to keep track of vehicles in the parking premises and can be helpful in case of anomalies like theft or vandalism when detected by our system. User interface provides users an easy to use system. Special efforts were made to ensure that the system is as intuitive and as easy to use as it can be. It automates most tasks needing minimal user intervention during the setup stages. System has the capability to be setup at any time to cater to changing conditions of the parking lot.

Anticipating the next phase of our FYP, essential tasks lie ahead to fortify the ParkEase's capabilities. Priority areas include the creation of a routing module and map generator to guide users with the shortest route upon entry. Additionally, the integration of cameras is imperative for seamless functionality.

In summary, the ParkEase successfully addresses challenges in parking management and security. While

certain aspects remain incomplete due to practical constraints, the achieved outcomes underscore the effectiveness of the implemented solutions.

Bibliography

- [1] N. Bibi, M. N. Majid, H. Dawood, and P. Guo, “Automatic parking space detection system,” in *2017 2nd International Conference on Multimedia and Image Processing (ICMIP)*, pp. 11–15, 2017.
- [2] Yujin and G. Xiaoxue, “Optimal route planning of parking lot based on dijkstra algorithm,” *2017 International Conference on Robots & Intelligent System (ICRIS)*, pp. 221–224, 2017.
- [3] K. R. Talha, K. Bandapadya, and M. M. Khan, “Violence detection using computer vision approaches,” in *2022 IEEE World AI IoT Congress (AIIoT)*, pp. 544–550, 2022.
- [4] E. P. Ijjina, D. Chand, S. Gupta, and K. Goutham, “Computer vision-based accident detection in traffic surveillance,” in *2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pp. 1–6, 2019.
- [5] H. Ghahremannezhad, H. Shi, and C. Liu, “Real-time accident detection in traffic surveillance using deep learning,” in *2022 IEEE International Conference on Imaging Systems and Techniques (IST)*, pp. 1–6, 2022.
- [6] P. Almeida, L. S. Oliveira, E. Silva Jr, A. Britto Jr, and A. Koerich, “Pklot—a robust dataset for parking lot classification,” *Expert Systems with Applications*, vol. 42, no. 11, pp. 4937–4949, 2015.
- [7] CarPark, “Cnrpark+ext dataset.” <https://universe.roboflow.com/carpark-sw6jq/cnrpark-ext-cnziv>, jun 2023. visited on 2023-10-07.
- [8] R. Szarka, P. Dakić, and V. Vranić, “Cost-effective real-time parking space occupancy detection system,” in *2022 12th International Conference on Advanced Computer Information Technologies (ACIT)*, pp. 506–510, 2022.
- [9] S. Rahman, M. Ramli, F. Arnia, A. Sembiring, and R. Muharar, “Convolutional neural network customization for parking occupancy detection,” in *2020 International Conference on Electrical Engineering and Informatics (ICELTICs)*, pp. 1–6, 2020.

- [10] L. Hu and Z. Liu, “Intelligent parking lot assistance system based on machine vision and a* algorithm,” in *2021 3rd International Symposium on Smart and Healthy Cities (ISHC)*, pp. 28–35, 2021.
- [11] S. Vibha, S. Shetty, and J. ShwethaR., “Heuristic solution for optimized path finding in smart parking system,” *International journal of engineering research and technology*, 2018.
- [12] P. Wu, j. Liu, Y. Shi, Y. Sun, F. Shao, Z. Wu, and Z. Yang, “Not only look, but also listen: Learning multimodal violence detection under weak supervision,” in *European Conference on Computer Vision (ECCV)*, 2020.
- [13] W. Sultani, C. Chen, and M. Shah, “Real-world anomaly detection in surveillance videos,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6479–6488, 2018.
- [14] L. Orenstein, N. Berges, Y. Fishelovich, D. Pinchuck, M. Edry, K. Nayman, N. Holzer, and R. Samen, “Sparkle: A vision-based smart parking system.” <https://github.com/dinap18/Sparkle>, 2020. Accessed on [date].
- [15] CVEDIA, “Smart parking.” <https://www.cvedia.com/smart-parking>, 2023. [Accessed: 7-Oct-2023].
- [16] K. Afshari, “Enhancing urban mobility via ar location-based parking navigation system.” <https://github.com/kimiaf1998/AR-based-Parking-Navigation>, 2023. Accessed on [date].
- [17] M.-R. Hsieh, Y.-L. Lin, and W. H. Hsu, “Drone-based object counting by spatially regularized regional proposal networks,” in *The IEEE International Conference on Computer Vision (ICCV)*, IEEE, 2017.
- [18] G. Amato, F. Carrara, F. Falchi, C. Gennaro, C. Meghini, and C. Vairo, “Deep learning for decentralized parking lot occupancy detection,” *Expert Systems with Applications*, vol. 72, pp. 327–334, 2017.
- [19] T. Hassner, Y. Itcher, and O. Kliper-Gross, “Violent flows: Real-time detection of violent crowd behavior,” in *2012 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, pp. 1–6, 2012.
- [20] H. Ghahremannezhad, H. Shi, and C. Liu, “Real-time accident detection in traffic surveillance using deep learning,” in *2022 IEEE International Conference on Imaging Systems and Techniques (IST)*, pp. 1–6, IEEE, 2022.